“Finite State Machines in Software Development: Creating a Learning Tool for the Rubik’s Cube”

Submitted August 2016, in partial fulfillment of

the conditions of the award of the degree MSc Software Engineering

Student: xxx xxx

School of Computing and Engineering

University of West London

I hereby declare that this dissertation is all my own work, except as indicated in the text:

Signature _____

Date _____/_____/_____

Table of Contents

Abstract

This report introduces the MSc Software Engineering dissertation project. The goal of the project is to create a learning tool for solving the Rubik's Cube, based on the Finite State Machines (FSMs). In order to achieve this goal, an existing solving strategy was selected and a number of FSMs was build based on the solving sequences, extracted from the strategy. The software, created for the project, bases its logic on those FSMs: they are used to demonstrate the solving sequences on the Cube's model to the user.

The report consists of six chapters. The first chapter introduces the project and describes its aims, objectives and methodology. The second chapter provides the background information about the areas, relevant to the project, namely the Rubik's Cube and Finite State Machines. The second chapter also contains the study of the previous work. The third chapter explains the selection of the solving strategy for the Cube and its adaptation into FSMs. The fourth chapter describes the development cycle of the project and gives examples of the code. The fifth chapter contains the evaluation of the application itself and the usage of FSMs in the project. The sixth chapter describes the future work on the project.

1. Introduction to the project

The planned project combines two main study areas: an application of Finite State Machines (FSMs) in the software development and the solution of the Rubik's Cube. Due to the fact that those areas do not directly overlap, the background of those two topics will be presented independently in the second section of the report. Current section introduces the project itself.

1.1. Aim and purpose of the project

The aim of the project is to create a software that uses the logic of FSMs to offer a learning tool for solving the Rubik's Cube. The software will offer the descriptions of the solving moves according to the one of the existing strategies. The implementation of the software will be done in C++ with the use of OpenGL library.

The main idea is to make the Rubik's Cube more accessible and attractive in the age of virtual games. This is important because while many children find mobile games interesting, playing them on the screen doesn't do much for the development of the fine motor skills. Those skills, however, are important part of the development and should not be ignored. A software that would transfer an interest of the child to the physical puzzle, could assist parents in the child's development by making the real puzzle something attractive rather than inflicted on the child. It is, however, not enough to just present the traditional descriptions of the solving moves due to the fact that their notations are oriented on the adults and won't be understood by the most children. So those notations will have to be visualised with the use of the 3D model of the Rubik's Cube in order to become understandable for the general public.

The use of FSMs was chosen because they provide a good way to organise the logic of the software so that it would be understandable to the people other than the ones who wrote the software in the beginning. The model, created on the basis of FSMs, can be presented as a set of states and transitions. This will make it structured and should ease the maintenance of the resulting software.

1.2. Objectives

The project has four objectives:

1. Analyse existing strategies for solving the Rubik's Cube to extract the solving rules for the individual positions;
2. Create FSMs that will be based on the extracted rules;
3. Create an application on the basis of the solution presented by using FSMs;
4. Analyse the effectiveness of FSMs as a basis of the software.

All of these objectives require the fulfilment of the previous ones, therefore they will be addressed in the report in the same order as in this list.

1.3. Methodologies

The main goal of the project is to create a software, so the full implementation cycle will be demonstrated during the work on the project. Due to the project's idea suggesting that all requirements should be collected before an actual implementation can be started, the waterfall model of the development process will be used. The selected model, as it can be seen in the next section, influences the schedule of the project.

The chosen programming language is C++ with the addition of the OpenGL elements for the Rubik's Cube's model development. The sub-chapter 4.3.1 gives the rationale for the selection of these tools. Due to the fact that there is an additional goal to investigate the possible advantages of the use of Finite State Machines in the software development, FSMs will be used to demonstrate the logical sequence of the application.

1.4. Schedule of the project

Twelve weeks are given to complete the project, from 30 May 2016 to 19 August 2016. During those weeks the defined objectives should be fulfilled and the report on the completed work (dissertation) of the size between 10 000 and 15 000 words should be written. The tasks are supposed to be spread between those weeks as follows:

Week 1: Analyse the existing strategies for solving the Rubik's Cube in order to select one of them and then extract the solving rules for the individual positions from it.

Week 2-4: Create FSMs that will be based on the extracted rules. Because the application is expected to demonstrate the solving rules separately from each other, each rule will become a basis for a separate Finite State Machine (FSM).

Week 4-5: Plan the design of the application: the user interface and the general structure of the application.

Week 6-10: Create an application on the basis of the solution presented by using FSMs.

Week 11: Test the created application.

Week 12: Complete the written part of the dissertation.

While the last week is intended for the writing of the report, it is mostly expected to be written during previous weeks, therefore week 12 will be used for the finalising of the report.

In the Gantt chart form, the schedule looks the following way:

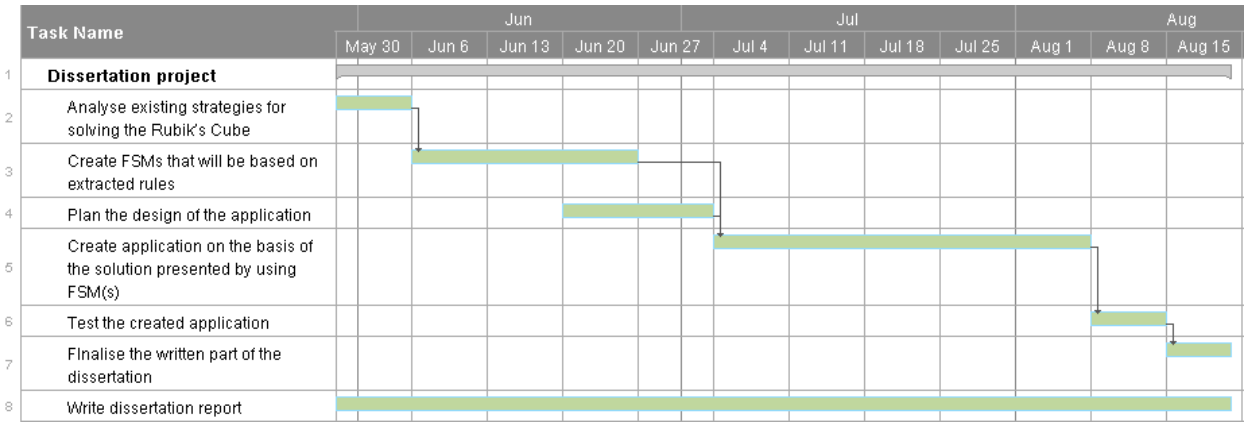

Figure 1. The Gantt chart

The chart demonstrates that there is one full development cycle planned in accordance with the waterfall model: the requirements collection (tasks 2-3), the design phase (task 4), the implementation phase (task 5) and the testing phase (task 6).

2. Background study

Current section introduces the history and some important information about Finite State Machines and the Rubik's Cube. Also this section contains the review of the previous work. The review is separated into three parts: the work on the use of FSMs in software development, on the search of the optimal solutions of the Rubik's Cube and on the combination of the first two subjects.

2.1. History study

This subsection provides the background information about Finite State Machines and the Rubik's Cube. These two topics are reviewed separately due to the fact that they do not overlap on their own.

2.1.1. Finite State Machines

A Finite State Machine (FSM), also called a Finite Automaton, is a model of a computational system (Houghton Mifflin Company). In 1943 in the paper "A Logical Calculus of Ideas Immanent in Nervous Activity" by Warren McCulloch and Walter Pitts mathematical logic was applied to the research of the information processing by neurons. This paper along with the Turing's paper on computability lay ground for the science of the computing (Lawson, 2003, p.23). Automata theory is the part of this science.

A FSM consists of three components: input string, sensor that reads symbols from the string and current state of the machine. It can be called a computer without memory (Goddard, 2008, p.3), because the actions of the machine only depend on its current state and ignore previous actions that put the FSM in it. The FSM has a finite number of states and there are two types of states that differ from the rest: the start state (only one) and the accept state (can be more than one). The accept state is the one where the FSM is supposed to finish after reading the whole string. If it stays in some other state after reading the given string then this string is not accepted by the FSM. This allows the FSM to act as a recogniser.

Finite state machines, being a theoretical concept, are not tied to any specific practical task. They can be used in both hardware and software development. Only deterministic FSMs can be used in hardware implementations, however. In software, a FSM can be used in string processing (text editors, DNA analysis), networks, computer games (AI controlled characters), lexical

analysis (compilers) and many other areas (Goddard, 2008, pp.44-48). Generally speaking, FSMs in the software development are used to aid with the design process (van Bergen, 2011). From this point of view, one can use them in any application area as long as the developer is sure about the requirements of the project and can build a model of the software. After the initial FSM is built, it can be expanded later if new requirements appear. Therefore, even if the model looks simple at the beginning it still can benefit from the use of the FSM in the future (Skorkin, 2011).

A FSM can be described in two ways. The first one is a graph, where nodes represent states and edges represent transitions between states (Wagner et al., 2006). It is usually called a state or a transition diagram (Wood, 1987, p. 95-96). This kind of description is easier to understand for humans, so it is often used for the demonstrations (as it can be seen in the academic papers, included in the previous work study).

The following example demonstrates the use of the transition diagram for the description of the FSM for the vending machine (one of the most often used in the literature examples). The said machine sells the drink at the cost $0.30 and accepts nickels, dimes, and quarters. It is also capable of returning the extra money when the drink is dispensed (Wright, 2005).

The building of the FSM is better to be done in steps from the simpler structure to the more complex one. Therefore, the starting FSM will look as follows:

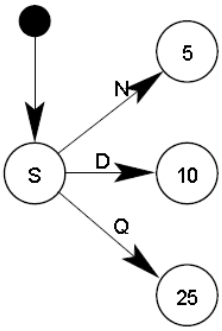

Figure 2 First step of the vending machine creation (Reproduced from Wright, 2005, p.3)

At this stage the FSM demonstrates the starting state (S) and the states, which could be reached by throwing accepted coins (N – nickel, D – dime, Q – quarter) into the vending machine. The middle states are named after the amount of money the machine received up to the current moment. This FSM can be expanded by adding states, reachable by throwing more nickel coins:

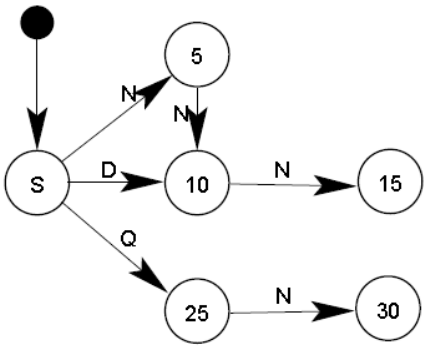

Figure 3 Second step of the vending machine creation (Reproduced from Wright, 2005, p.3)

The FSM can be further expanded by addition of all possible coins input:

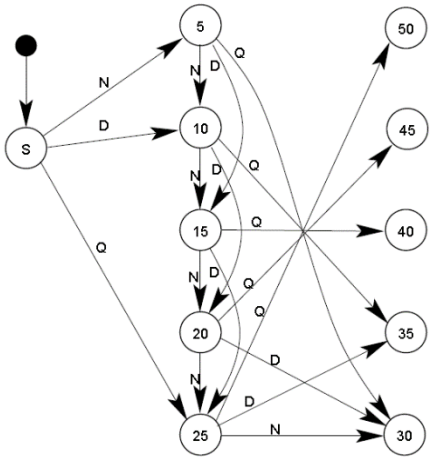

Figure 4 Third step of the vending machine creation (Reproduced from Wright, 2005, p.4)

The FSM should return the money in case the user threw in more than 30 cents, so the FSM from the third step can be optimised. In order to do this the states, exceeding 30, can be merged with the '30' state with the addition of the returned value:

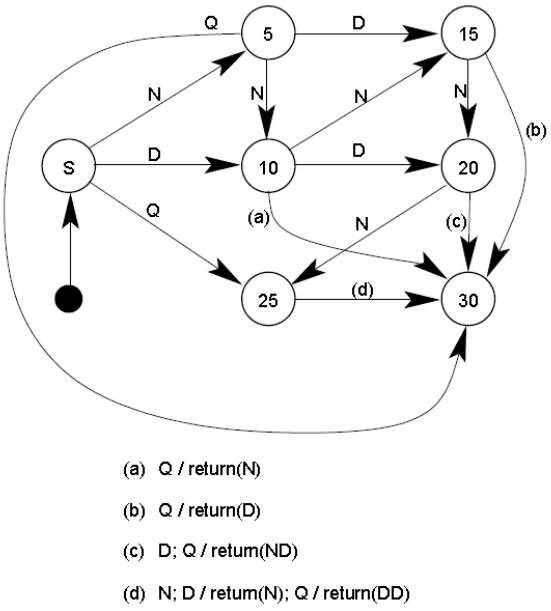

Figure 5 Optimised FSM (Reproduced from Wright, 2005, p.6)

The graphical representation is easy to follow, but this is only true when the number of states and transitions between them is relatively small. The Figure 4 demonstrates how a large number of states can hinder the visual perception of the FSM. In this case, another type of definition, which is more formal, can be used. This one may be harder to grasp for humans, but it is well adapted for the writing of the code, based on the FSM. This definition includes the following elements (Goddard, 2008, p.9):

- Q: a finite set of states;
- ∑: an alphabet of possible input symbols;
- q0: a start state;
- T: a subset of Q, consisting of accept states;
- δ: a transition function, $\delta: Q \times \sum \rightarrow Q$. It maps a state and symbols from the alphabet to a state. If δ(a,1) = b it means that if the FSM reads 1 while being in the state a it will go to the state b.

A Finite State Machine, described by those notations is called a deterministic FSM. This means that the unique actions for the each state are completely specified. The FSM can also be nondeterministic. This means that for the each state there can be more than one possible transition for the same symbol.

Another important characteristic divides FSMs into two types: Moore and Mealy machines. While moving from state to state a FSM performs actions, which can be of four different types:

entry, exit, input and transition actions. Entry actions are done when the machine enters the current state, exit action – when it leaves the state. Transition action is performed during the state change. Input action can potentially not be related to the specific state and is performed when some condition is true. It is the only action that can be done without the state change. Effectively, Moore machine generates only entry actions and Mealy machine generates only input actions.

2.1.2. The Rubik's Cube

Puzzles are single-player games that are enjoyable to play and they should have a solution that gives their user a satisfaction in reaching it (Kendall, Parkes and Spoerer, 2008). There is a lot of different types of puzzles, such as a sliding puzzles (15 Puzzle), a dissection puzzles (Tangram), a tour puzzles (various mazes), a combination puzzles (the Rubik's Cube) and many more.

The Rubik's Cube was created in 1974 by Ernö Rubik, Hungarian professor of architecture. Originally it was not intended as a puzzle, but as a model, where parts could be moved without destruction of the whole structure (Davis, 2014). When Rubik created his model, he marked each side in different colour and rotated its parts. The model worked well, however it took him about a month to return it in its original state. Rubik showed it to his students, thinking that this cube could be used for the lessons on group theory and spatial relationships. After this he decided that the cube could be interesting to the general public and tried to bring it to the market. First try was made in 1977 and the trade name of the cube then was "The Magic Cube". However, the puzzle became popular only in the eighties, when it was advertised in the toy fairs outside Hungary. It was rebranded as "The Rubik's Cube" at that time. The peak of the puzzle's popularity dropped significantly towards the end of the nineties, but was somewhat restored in 2004, when Mark Longridge reincarnated the ""cube-lovers" mailing list" (Rokicki et al., 2013, p.1084). Now it still remains one of the most popular puzzles.

Classical and the most famous Rubik's Cube has the dimensions 3×3×3. The World Cube Association, which currently organises the competitions for the Cube solving, recognises different types of the Cube (from 2×2×2 to 7×7×7) same as the different solving conditions (for example, blindfolded). In those competitions the winner is decided on the basis of the average time of five tries, whilst the singular fastest time is also recorded. Currently for the Rubik's Cube the best average solving time equals 6.54 seconds (Feliks Zemdegs) and best single try equals 4.90 seconds (Lucas Etter) (World Cube Association). The current best moves record is 19 (single) and 24 (average). To compare, the general solving strategies fall between 50 and 100 moves (Korf, 1997, p.700).

From the mathematical point of view, the Rubik's Cube is closely tied to the group theory. The Cube can be used to explain it and the latest academic research on the Rubik's Cube solution (which demonstrates that it can be solved in 20 or less moves) used group theory for proving. From more general point of view, solving the Rubik's Cube helps to develop strategic thinking and spatial intelligence (Vega Society, 2006). Also the sliding of the Cube parts can have a calming effect (Attwood, 2006, p.173). All this makes the Rubik's Cube an interesting research object.

2.2. Previous work

In order to study the work that was previously done on the selected topics, several academic papers were selected. Those papers can be divided into three groups according to their topic: finding solution for the Rubik's Cube, developing software with the use of Finite State Machines and using Finite State Machines to solve the Rubik's Cube. The latest topic is the combination of the first and the second.

The papers on the topic of finding solution for the Rubik's Cube were selected to demonstrate the different approaches to the search of the optimal Rubik's Cube solution as well as the evolution of those approaches. The list of the papers in the chronological order goes as follows:

- Finding optimal solutions to Rubik's cube using pattern databases (Korf, 1997);
- An evolutionary approach for solving the Rubik's cube incorporating exact methods (El-Sourani, Hauke and Borschbach, 2010);
- Algorithms for solving Rubik's cubes (Demaine *et al.*, 2011);
- The diameter of the Rubik's cube group is twenty (Rokicki *et al.*, 2013).

The third paper in the list offers theoretical analysis of the Rubik's Cubes' solutions while the rest of the papers use more practical approach to discover the required number of moves and/or solve the puzzle.

The next group of papers demonstrates the usage of Finite State Machines in different fields:

- Gesture modeling and recognition using Finite State Machines (Hong, Turk and Huang, 2000)
- A Finite State Machine approach for modeling and analyzing restful systems (Zuzak, Budiselic and Delac, 2011).
- Computing game design with automata theory (Qureshi *et al.*, 2012);

- Finite State Machine based vending machine controller with auto-billing features (Monga and Singh, 2012);

Two papers from this list represent fields that are traditionally used as the examples of a FSM applications (Qureshi *et al.*, Monga and Singh): computer games and vending machine controller. Another two papers in the list apply FSMs to a less frequently used areas, namely the modelling of distributed systems and gesture recognition.

The last group includes two papers that use FSMs to solve the chosen problem and also use the Rubik's Cube problem for the demonstration of their solution. Those papers are older than the papers in the first two groups and were chosen because of the lack of the more recent papers on the similar combination of topics. The selected papers are:

- Diversity-based inference of finite automata (Rivest and Schapire, 1987);
- Pruning duplicate nodes in depth-first search (Taylor and Korf, 1993).

2.2.1. Finite State Machines

This subsection contains the overview and the analysis of the four papers about the use of Finite State Machines in the software development.

2.2.1.1. Overview

The selected papers in this group were published in the time period from 2000 to 2012. One of the papers (Hong, Turk and Huang) was published as a part of the proceedings of the fourth IEEE international conference on automatic face and gesture recognition, and the rest of the papers was published in three different journals: Journal of Web Engineering (Zuzak, Budiselic and Delac), International Journal of Multidisciplinary Sciences and Engineering (Qureshi *et al.*) and International Journal of VLSI design and Communication Systems (Monga and Singh).

The main idea of all papers is the creation of the software that would be based on Finite State Machines, but the application area of the software is different for each paper. Hong, Turk and Huang suggest to apply FSMs to the gesture recognition, which should allow the real-time performance because of the computational efficiency of the FSM. Monga and Singh propose to use FSMs in a vending machine in order to reduce the hardware and produce faster response. Qureshi *et al.* apply FSMs to the computer game design. Zuzak, Budiselic and Delac use FSMs in the modelling of the RESTful systems.

Due to the similarity of the goals of the papers, the steps that authors took in order to achieve them are also similar. All papers start with the description of the application area and the outline of the logic, which authors want to implement in the software. After that the created FSMs are described in the relation to the previously defined logic followed by the practical implementation (software development).

What differs is the completeness of the software development, because Qureshi *et al.* only describe the design of FSMs and are yet to develop planned game. Other papers go further and present the prototypes of the planned software. All papers use graphs to demonstrate the designed FSMs, but some of them (Qureshi *et al.*, Zuzak, Budiselic and Delac) also provide the formal description of the automata (alphabet and states).

2.2.1.2. Critique

All reviewed papers in this group have a goal of implementing software with the use of FSMs. They all discuss the planned solution in theory, and all of them, except Qureshi *et al.*, implemented their theoretical model on practice at the time of the paper submission. Qureshi *et al.* only present the FSMs in the paper, but make future plans of the development of the designed game. Following review of the strengths and the weaknesses was made from the point of view of the use of FSMs.

2.2.1.2.1. Strengths

Authors of all papers put their work into context by reviewing the work, which is related to their cause. This makes the papers more valuable for future researchers as it helps them to evaluate the presented work. This is, however, an objective strength, because subjectively from the point of view of the planned project most of the related work in the reviewed papers is not relied to the FSMs. This can be viewed as a display of the relatively low use of FSMs in the software development and may be worth exploring as a potential gap in the knowledge.

All reviewed papers describe the logic that authors used to create FSMs and present those automata in the form of graphs. Some (Qureshi *et al.*) go into more detail than others (Hong, Turk and Huang), but all papers allow clear view of the design. This makes them valuable in both academic and practical sense.

2.2.1.2.2. Weaknesses

While the strengths at least partly can be combined and discussed together, the weaknesses are better to be discussed separately for each paper. In general, the reviewed papers tend to lack formal description of the created Finite State Machines. In order to make the distinction of papers easier, their headings are provided at the beginning of the paragraphs.

Computing game design with automata theory. While authors describe the created FSMs in the detail, the paper lacks reasoning of selecting FSMs as a method in the first place. This does not diminish the value of the paper as a source of the information about the development of the software with the use of FSMs, but does make it less valuable in an academic sense. Also one can perceive as a weakness the fact that the paper states that the nondeterministic FSMs, described in the paper, cannot be directly used for writing of the software but does not offer any solution for transferring them into the deterministic FSMs. This, however, may be outside out the paper's scope because authors have planned the implementation of the game as their future work.

Finite State Machine based vending machine controller with auto-billing features. As a weakness of this paper one can name the lack of a formal description of the used FSM. Authors provide the graph form along with its description, but do not state explicitly an alphabet of the designed FSM. Authors design a vending machine, so the possible input will be naturally restricted by the existing buttons and accepted types of coins and banknotes, so this may be not as important as it could be in the design of the purely computer-based software. However, the paper still could benefit from an alphabets' inclusion.

Gesture modeling and recognition using Finite State Machines. While authors give the detailed description of the implemented algorithm and explain how FSMs are used in it, they only provide the examples of the individual FSM without any formal description. This makes it harder to evaluate the correctness of the use of FSMs in this study.

A Finite State Machine approach for modeling and analyzing restful systems. This paper is a continuation of the authors' previous work, so it covers more of its selected topic than the rest of reviewed papers in the group do for their topics. However, while authors provide a list of the previously existing approaches to the modelling of the RESTful systems and mention their drawbacks, they do not provide a comparison between the presented solution and the other approaches. Generally speaking, the paper could benefit from such addition.

2.2.2. The Rubik's Cube

This subsection contains the overview and the analysis of the four papers about the solutions of the Rubik's Cube.

2.2.2.1. Overview

The reviewed papers were published in the time period from 1997 to 2013. All papers, except for one (Rokicki *et al.*) have been published as a part of the conference proceedings: Algorithms – ESA 2011 (Demaine *et al.*), the Fourteenth National Conference on Artificial Intelligence (Korf) and EvoApplications 2010 (El-Sourani, Hauke and Borschbach). The paper "The diameter of the Rubik's cube group is twenty" was published in SIAM Journal on Discrete Mathematics in 2013. First presentation of this idea by the same authors was published in 2010, however.

The selected papers are united by the common goal: provide an algorithm for finding solution of the randomly scrambled Rubik's Cube. Supposedly because of the Rubik's Cubes' connection to the group theory, all papers rely on it in order to provide the solution. Specific methods they use, however, are different. Also all papers except for one (Demaine *et al.*) look for the solution of the 3×3×3 Cube. Demaine *et al.* search for the more general algorithms for the n×n×n and n×n×1 cubes. Due to this fact it makes more sense to summarise this paper separately from others.

Demaine *et al.* start with finding the diameter through upper and lower bounds for the n×n×1 cube. In order to do this, they introduce the notion "cubie cluster (x,y)", which means a set of the locations, reachable for a cubie (one of the smaller cubes that make the Rubik's Cube), located at (x,y). While finding a sequence of moves for solving the specific cluster without affecting other clusters, authors are able to define the asymptotic upper bound of the algorithms' complexity function as $O(n^2/\log n)$. After that they produce the asymptotic lower bound $\Omega(n^2/\log n)$. Having found this, authors proceed to the n×n×n cube. Same as in the previous case, the idea of solving the cluster without affecting other clusters is used. With the use of the similar technique authors are able to prove the same upper ($O(n^2/\log n)$) and lower ($\Omega(n^2/\log n)$) bounds for the n×n×n cube.

Work that was done in the rest of the reviewed papers is more practical. One of the remaining papers (El-Sourani, Hauke and Borschbach) applies an evolutionary algorithm to a process of solving the given scrambled Cube according to the Thistlethwaite's approach. The main idea of this approach does as follows: the problem of solving the Cube is divided into the four subproblems. Suggested in the paper an evolutionary algorithm multiplies the given Cube and tries to solve the first subproblem for each copy. After a sufficient amount of copies is solved, a predefined number of the best solutions is selected for the next stage, where the second subproblem

will be solved. The algorithm continues this cycle until all subproblems are solved. According to the Thistlethwaite's approach the Cube should be solved in 45-52 moves and the results of the presented algorithm correspond with this number. This is, however, not the best possible solution and authors name a number of the possible improvements as a potential future work. From the point of view of the dissertation project, though, this approach may be suitable as a basic solving strategy.

The other two papers rely on the pattern databases in order to solve the Rubik's Cube. Unlike the previous paper, authors of these papers are trying to find the optimal solution of the Cube. Korf in 1997 put the median solution length at 18 moves and was the first to apply the pattern databases to the Rubik's Cube problem. Rokicki *et al*. adapted Korf's method and, having more computational power at their disposal, proved that any scrambled Cube can be solved in no more than 20 moves. Computational power is important in this case because the pattern databases are essentially the precomputed tables containing solving moves for the known states of the Cube, so their use might be compared to the brute-force search. Therefore, more available CPU power and more extensive memory mean an analysis of more possible states in a shorter time. However, Rokicki *et al*. also improved the search algorithm in addition to the increased computational power. They suggested that if the top layer of the Cube is solved from the start then the solution of this Cube can also be applied to the Cubes that differ from it only by a move of the bottom layer. This operation, called prepass, allowed authors to make the search process more effective.

2.2.2.2. Critique

The main goal of the reviewed papers is either to create an algorithm that would find a solution for the randomly scrambled Rubik's Cube or to provide the information about such solutions. All papers manage to reach their goals, theoretical as well as practical. From the point of view of the planned project, however, papers lack extended information about the way humans solve the Cube, mostly concentrating on the computer search of the optimal solution.

2.2.2.2.1. Strengths

All reviewed papers discuss the previous and the parallel work on their topic, which puts authors' own work into the context. Also in all cases authors provide the detailed description of their work so that future researchers would be able to follow it. Rokicki *et al*. and El-Sourani,

Hauke and Borschbach outline the planned future work, which also makes it easier to follow for other researchers.

2.2.2.2.2. Weaknesses

As with the FSMs papers group it seems more suitable to review the weaknesses of the papers separately. The headings are also provided at the beginning of the paragraphs.

An evolutionary approach for solving the Rubik's cube incorporating exact methods. Authors presented the previous work on Rubik's Cube solution using an evolutionary approach same as other computational approaches. However, they present no comparison between the results of those approaches and their work. While the description of the method is clear and allows other researchers to repeat the process for themselves, a lack of such comparison still makes it harder to evaluate the presented method.

Finding optimal solutions to Rubik's cube using pattern databases. This paper is the oldest in the group and therefore its author had less computational power available than others. However, in my opinion, it still would be possible and more practical to generate more than ten scrambled Cubes for the experimental stage. This would potentially increase the diversity of the results and make them more reliable.

Algorithms for solving Rubik's cubes. The reviewed paper lacks the conclusion part. The main goal of the paper is to present the theorems about the upper and the lower bounds of the Cube and therefore any practical applications of those theorems are outside of the papers' scope, so the missing conclusion does not really interfere with the papers' aims. However, it still influences readers' perception of the paper so it may be viewed as a weakness.

The diameter of the Rubik's cube group is twenty. Authors give a detailed explanation of their algorithm, but provide much less details about its practical work. This does not diminish an academic value, but makes the paper less obvious, so it probably could benefit from an additional information about the achieved results.

2.2.3. Combining Finite State Machines and the Rubik's Cube

This subsection contains the overview and the analysis of the two papers which combine the Rubik's Cube and the use of Finite State Machines.

2.2.3.1. Overview

Two papers in this group were published in 1987 (Rivest and Schapire) and 1993 (Taylor and Korf). The second paper, "Pruning duplicate nodes in depth-first search" became the base for the doctoral dissertation of one of its authors (Taylor) in 1997. Both papers were presented in the conferences, 28th Annual Symposium on Foundations of Computer Science (Rivest and Schapire) and AAAI National Conference (Taylor and Korf).

Both papers use the Rubik's Cube solving problem for the demonstration of methods, presented in the paper. Finite State Machines play different role in those methods, however. Taylor and Korf use the FSM to execute the pruning of the duplicate nodes to lessen the complexity of the depth-first search because FSMs allow the efficient use of time and space. Rivest and Schapire make the creation of the method to analyse FSMs (inferring structure from its input/output behaviour) the goal of their paper.

Taylor and Korf present a method that contains two steps. Firstly one needs to perform a breadth-first search of the problem space in order to define paths, which generate duplicate nodes. Those paths can be presented as strings. After that a FSM, which recognises paths that were found during the first step, can be created. If this FSM recognises any path, the node at its end can be safely dropped. Authors apply this method to several puzzles, including the Rubik's Cube, managing to simplify the search in all cases.

Rivest and Schapire access their goal from the rather formal point of view. They start with the definition of the finite state automaton and proceed to the definitions of the different elements and the notions of it. The main term they use is a test, which is an action followed by a predicate. Tests can succeed or fail depending on the state of the FSM. The sets of different tests, simulating the FSM in question are used to describe the inference procedure that authors created. The procedure is based on the number of theorems, defined and proved in the paper. Authors used the procedure in several environments (FSMs), including the Rubik's Cube and received the satisfactory results.

2.2.3.2. Critique

Authors of both reviewed papers used the Rubik's Cube to demonstrate the work they've done. The main ideas of the said work is, however, different. Rivest and Schapire created a method for inferring the FSM from its input/output and Taylor and Korf used FSMs as a tool for removing the duplicate nodes during the depth-first search.

2.2.3.2.1. Strengths

Authors of both papers provide rather detailed description of the methods they are presenting. It allows others to follow the work they've done and refer to it in their own work if necessary, which increases the value of the reviewed papers in an academic sense. Authors, in turn, recognise the work that was done before them, which also can be named as a strong side of the papers.

Taylor and Korf provide a reasoning for choice of the FSM as a part of their method, this makes the paper more trustful in terms of the validity of the provided solution. Also authors recognise the limitations of FSMs, which makes it easier to define if the method is applicable in a specific situation or not.

2.2.3.2.2. Weaknesses

Unfortunately, neither paper provide an exact description of the practical application of the presented methods, which puts anyone trying to reproduce those experiments at a disadvantage. Rivest and Schapire refer to their other paper for the detailed description of their experimental results, but it also doesn't provide many details on the topic. Perhaps one of the reasons for this is the complexity of the experimental data.

2.2.4. Conclusion

This review of the previous work demonstrates several points. FSMs are not widely used in the software modelling according to an academic sources. However, they were successfully used in the reviewed papers, for example the switching speed of the vending machine, designed by Monga and Singh (2012) was 9.3 ns when the other solutions had 12.53 and 300 ns switching speed. The gesture recognition method, offered by Hong, Turk and Huang (2000), reduces the computational complexity compared to the application of hidden Markov's models (HHMs) and does not require the predefined model. This demonstrates that FSMs are the valid tool to be used in the software modelling.

The reviewed papers about the Rubik's Cube's solutions demonstrate that not only the general public, but the academics as well are interested in this puzzle. This makes it a good choice as an application area for the project.

As for the topic of this dissertation proposal, usage of FSMs for the modelling of the Rubik's Cube solving moves, no absolutely fitting academic papers exist. The reviewed papers that

present a combination of those two topics demonstrate methods, where FSMs are used and apply those methods to the Rubik's Cube solutions. Probably due to the fact that the peak of this puzzle's popularity fell on the eighties those papers are not recent: 1987 (Rivest and Schapire) and 1993 (Taylor and Korf). Rivest and Schapire created a method for inferring the FSM from its input/output and Taylor and Korf used FSMs as a tool for removing the duplicate nodes during the depth-first search. Those paper show that the selected for the project topics can be combined as well as demonstrate the fact that it is not widely done. Therefore, this report can potentially introduce something new, even if the chosen areas were already studied for a long time.

3. Strategy for solving Rubik's Cube

In this chapter the solving strategy for the Rubik's Cube will be selected and adapted for the design of the Finite State Machines. The creation of the FSMs will be explained based on the selected examples and the rest of FSMs can be found in Appendix 1.

3.1. Strategy selection

The Rubik's Cube exists since 1974 and during this time a lot of thought was given to the search of the solution of the puzzle. A number of solving strategies was developed, some of them for the beginners and some of them for the advanced players. Each one of those strategies can be sufficient for the project, since they all offer the complete description of the Cube's solution process. The following sub-sections analyse the selected strategies against the list of the criteria.

3.1.1. History of the development of the strategies

Erno Rubik spent one month trying to solve his Cube after he created it in 1974. He never made any notes, however (Slocum *et al.*, p. 22). When the interest to the Cube grew, people started to discuss solutions, but everyone used their own notations, usually based on the Cube's colours or the coordinate system. The strategies, described by those notations, were often unclear and not exactly fit to be used by anyone except from their creator.

In 1979 David Singmaster created the set of notations that became a standard and allowed to organise the work on the Cube. He came up with the set of the directions: Front, Back, Left, Right, Up and Down (Slocum *et al.*, p. 26). This allowed him to define the faces of the Cube as well as its movements. In 1980 Singmaster expanded his manuscript to 75 pages and included a description of the step by step solution. Since then a lot of other solutions were presented: in the years of the Cube's hype (1979 - 1983) 325 books were published (Slocum *et al.*, p. 35).

Currently existing solution methods can be roughly separated into the two groups: methods for the beginners and methods for the speedcubing (Slocum *et al.*, p. 74). The latter group requires a lot of memorising and, as the more advanced, falls outside of this work's scope. Most of the solving strategies are separated into a number of steps and can be organised according to the number of the solving algorithms they use.

3.1.2. List of the strategies

The work on this project requires a selection of the one of the existing strategies in order to adapt it into the application and demonstrate it to its user. The analysed strategies are:

1. The Rubik's Cube official website (Five steps) https://uk.rubiks.com/blog/how-to-solve-the-rubiks-cube. Technically, there is a six steps on the website, but the first one contains the description of the notations and doesn't serve as a part of the strategy itself.
    - First step: Solve the white cross on the top (Up) face;
    - Second step: Solve the white corners;
    - Third step: Solve the middle layer;
    - Fourth step: Put the yellow cubies on the correct layer (ignoring correct positions);
    - Fifth step: Position yellow cubies correctly.
2. The "How To Solve A Rubix Cube" website (Seven steps) https://how-to-solve-a-rubix-cube.com/.
    - First step: Solve the white cross on the top (Up) face;
    - Second step: Solve the white corners;
    - Third step: Solve the middle layer;
    - Fourth step: Make the yellow cross (ignoring fitting of other colours);
    - Fifth step: Fix the position of yellow edges;
    - Sixth step: Position yellow corners (ignoring fitting of other colours)
    - Seventh step: Fix the position of yellow corners.
3. Jerry Slocum *et al*. (2009) (Six steps)
    - First step: Solve the one layer's corners;
    - Second step: Solve edges of the same layer;
    - Third step: Solve the remaining corners;
    - Fourth step: Solve the opposite layer's edges;

- Fifth step: Position the remaining edges;
- Sixth step: Orient the remaining edges.

In order to select one strategy it is necessary to define a number of the criteria. Those criteria will show how well the strategy fits for the purposes of the project. During the analysis the strategies will be named according to their numbers in the list above.

3.1.3. List of the criteria.

In order to fit the teaching purposes of the planned application, the strategy should be:

1. Easy to understand;
2. Well-structured;
3. Be applicable to any state of the Cube (completely scrambled or half-solved).

Since the strategy will be used not in the human-human teaching but in the algorithm of the application, it also needs to be:

1. Easy to adapt as an automated algorithm (minimum intuitive moves);
2. Described in the detail (to simplify adaptation).

3.1.4. Evaluation of the strategies against criteria

To make the analysis more evident, it is displayed in a table form.

Table 1 Comparison of the strategies against the criteria

Criteria/Strategies	**First strategy**	**Second strategy**	**Third strategy**	**Conclusion**
Easy to understand	The strategy is offered mostly in the form of the videos (one per each step), some of the solving sequences are	The strategy offers a number of the solving algorithms for each step. The algorithms are illustrated with the image of the	The strategy offers the step by step solution and lists the used solving sequences. The way of demonstration	The first and the third strategy offer step by step illustration, which makes it easier to learn the

	illustrated by pictures, demonstrating the states of the Cube. Videos may be hard to navigate in order to get to the specific part of the step, however.	Cube at its starting state, the sequence of states is not demonstrated. This strategy offers more situation specific algorithms.	however, might be viewed as confusing because of the necessity to display the unseen cubies on the paper.	solving sequences. The second strategy, however, covers more specific states of the Cube per step.
Well-structured	Divided into five steps, the order of steps is clearly understandable and follows one direction (top, middle, bottom layer).	Divided into seven steps, the order of steps is clearly understandable and follows one direction (top, middle, bottom layer).	Divided into six steps, the order of steps is clearly understandable, but doesn't follow one direction (top layer, bottom layer, middle).	The first and the second strategy organised more logically than the third.
Applicable to any state of the Cube	Can be applied to the partly solved Cube.	Can be applied to the partly solved Cube.	Can be applied to the partly solved Cube.	All strategies are similar.
Easy to adapt as automated algorithm	The strategy requires some additional work on the first step, which is mostly described as intuitive.	The strategy requires some additional work on the first step, which is mostly described as intuitive. Provided situation-	The strategy requires some additional work on the first step, which is mostly described as intuitive.	The second strategy requires less additional work on the creation of the situation-specific

		specific solving sequences simplify this work, however.		solving sequences.
Described in detail	Algorithms are described well enough to be adapted into the FSM.	Algorithms are described well enough to be adapted into the FSM.	Algorithms are described well enough to be adapted into the FSM.	All strategies are similar.

The analysis shows that each of the selected strategies can be deemed suitable. However, the second strategy provides more solving sequences, adapted for the individual states of the Cube than the other two strategies. Therefore it will be the one selected for the project.

3.2. Strategy adaptation

With the solving strategy being selected, it is now possible to design the Finite State Machines for this strategy. This sub-section first describes the notations that will be used in the FSMs design and then provides the example of their creation based on the solving process for the very first cubie during the first step of the strategy. Since the Rubik's Cube is a very complex puzzle that allows a big number of possible moves, each solving sequence will be described with the separate Finite State Machine. This will help to optimize time, spend on using the FSM during the work of the application.

3.2.1. FSM structure

For the purposes of the project the deterministic FSMs are required, due to the fact that they will be based on the Rubik's Cube's states and this puzzle can only be changed into the one unique state with any one move from any given state. The deterministic FSM can be described with the following five elements:

- Q: a finite set of states;
- $\sum$: an alphabet of possible input symbols;
- q_0: a start state;

- T: a subset of Q, consisting of accept states;
- δ: a transition function, $\delta: Q \times \sum \rightarrow Q$. It maps state and symbols from the alphabet to the state. If δ(a,1) = b it means that if the FSM reads *1* while being in the state *a* it will go to the state *b*. (Goddard, 2008)

Therefore, in order to describe each FSM it is necessary to define those five elements. All FSMs will be designed with the use of the same logic, described in the next sub-section.

3.2.2. Main idea

Before defining the specific FSMs, it is necessary to define the general rules for the creation of the five elements of the FSM. *T* and q_0 belong to the set *Q* and *δ* requires *Q* and ∑ to be defined, so at the beginning ∑ and *Q* need to be defined. In order to avoid confusion, notations have to be described before defining the elements of the five-tuple.

3.2.2.1. Notation's description: States (Q)

Since the FSM describes the sequence, which moves the Rubik's Cube from the one state to some other state, the states of the Cube can be used as the states of the FSM. Also, due to the fact that the FSM is based on the specific solving sequence, this set of states is limited to the states, which are relevant to the given sequence. Those states must be clearly described in order to make the FSM understandable.

Solving sequences move some given cubie from the one position to another. This means that the position of that cubie can be used to define the state of the Cube and, accordingly, the FSM. However, some sequences may include moves, which do not change the position of the selected cubie. Therefore, the positions of the additional cubies must be added. To simplify the description of the state, only the minimal amount of cubies must be added, preferably one (if possible). The selected cubie must change the position when the main cubie stays in place.

To define the position of the cubie, two things are need to be known: the physical place and the orientation. Since the orientation of the Cube doesn't change during the sequence, a coordinate system can be used for the definition of the physical place of the cubie. According to this system, each cubie has a set of three coordinates (x, y, z) and each of those can have a value from 1 to 3. So, the top right corner of the front face has coordinates (3, 3, 1).

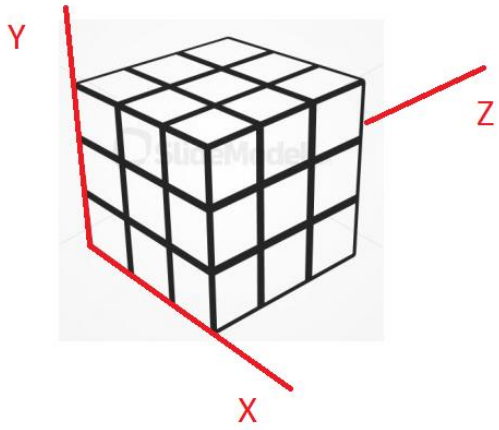

Figure 6 The coordinate system for expressing the cubie's position

Each cubie can have up to three visible differently coloured sides. If all three sides are visible, the cubie is called corner and it can be oriented in three different ways. The cubie with two visible sides is called edge. Edge can be oriented in two different ways. Since the different orientation of the cubie means the different state of the Cube itself, states are defined not only by the physical position of the cubies, but also by their orientation. Each solving sequence solves the cubie from the selected side of the Cube. This means that the colour of that side can be used to define the orientation of the cubie: if the cubie is positioned in a way that the colour that is being solved is on the front face of the Cube, then the orientation is defined by the letter F. Letters used are the same as the ones, used for the turn's notations (next sub-section). The only exception from that rule will be the edges in the middle layer of the Cube, where two colours are solved simultaneously. In this case the colour that is being solved will be defined as the colour of the front face of the Cube.

According to this, the state of the FSM will be defined by the set of the following 5-tuples:

$$St = (S, x, y, z, C),$$

where S can equal 1 for the cubie that is being solved or 0 for additional cubie,

x, y and z define the current coordinates of the cubie,

C defines face of the Cube, where selected colour is currently situated.

If solved cubie changes positions with every move in the sequence, then the set will consist of the only one 5-tuple.

3.2.2.2. Notation's description: Alphabet (∑)

Solving moves for the Rubik's Cube do not involve the changing of the orientation of the Cube in the middle of the sequence. This means that the possible input to the FSM consists of the rotations of the Cube's sides. Those rotations have the widely accepted notations that can be used. Those notations are based on the turned face:

- Clockwise turn (90 degrees):
  - U (up);
  - D (down);
  - L (left);
  - R (right);
  - F (front);
  - B (back).
- Counter clockwise turn (90 degrees):
  - U' (up);
  - D' (down);
  - L' (left);
  - R' (right);
  - F' (front);
  - B' (back).
- 180 degrees turn:
  - U2 (up);
  - D2 (down);
  - L2 (left);
  - R2 (right);
  - F2 (front);
  - B2 (back).

Therefore, for any FSM $\sum$ is a set of those notations: $\sum$ = {U, D, L, R, F, B, U', D', L', R', F', B', U2, D2, L2, R2, F2, B2}.

3.2.3. General description of FSMs

Q: $\{\{St_i\}_j$: St = (S, x, y, z, C), *i* is a number of cubies, required to define state *j*. *j* can have a value from 1 to N+1, where N is a number of states that the Cube takes during the sequence (including starting and ending ones). $\{St_i\}_{N+1}$ = (0, 0, 0, 0, 0) (any state of the Cube, unrelated to the sequence)}

q_0: State of the Cube, required by the algorithm before the sequence can be started.

T: State of the Cube that it should come to after the completion of the sequence (can be only one).

∑: {U, D, L, R, F, B, U', D,' L', R', F', B', U2, D2, L2, R2, F2, B2}.

δ: If the input changes state of the Cube into one, belonging to Q, function leads to that state. Else it leads to the state $\{St_i\}_{N+1}$.

3.2.4. Adaptation procedure

As a learning tool, the planned application needs to provide the step by step instructions for each cubie. This means that first step of the strategy, for example, will be separated into four sub-steps: four cubies of the cross. The selected strategy, same as others, doesn't provide the detailed description of the moves, required for the solving of the white cross during the first step. The algorithms, provided for the rest of the steps, also do not cover every possible state of the Cube. Therefore, in order to create a teaching application on its base, it is necessary to determine the required sequences beforehand.

In order to do that, first a list of the possible positions of the cubie that is being solved needs to be determined. Then for the each item of the list a solving sequence can be created. These sequences will then serve as a basis for the FSMs.

The strategy doesn't determine the order, in which the colours should be solved during each step. The adapted strategy will do this in order to make the solving process more structured. For example, for the first step the following scheme will be adapted: White side is the UP face, yellow is the DOWN face. Strategy doesn't specify the colour of the middle faces, but this algorithm assumes, for the sake of the simplicity, that the FRONT face is blue, the BACK face is green, the RIGHT face is orange and the LEFT face is red. Due to the structure of the Cube, the colours are paired: blue is always opposite from green and so on. This means that the user will need to confirm the colours of only three faces: top, front and right. The cubies of the cross will be solved in the following order: front (white-blue), left (white-red), back (white-green), right (white-orange). Turning the Cube in a way that will put cubie that is being solved in the front will help to minimise the number of FSMs, required for this step, since the relative positions of the cubies against each other will repeat themselves.

The white-blue cubie therefore will be the very first being solved. At this stage none of the edge cubie are solved, so required cubie can be in any of the 12 positions. This, along with the two possible orientation in each position, leads to 24 possible states. Those states for the white-blue cubie and their solving sequences are demonstrated in the following table.

Table 2 The starting states and the solving sequences for the white-blue cubie

№	Starting state	Solving sequence	Ending state
1	(1, 2, 3, 1, U)	-	(1, 2, 3, 1, U)
2	(1, 2, 3, 1, F)	F R U	(1, 2, 3, 1, U)
3	(1, 3, 2, 1, R)	F’	(1, 2, 3, 1, U)
4	(1, 3, 2, 1, F)	R U	(1, 2, 3, 1, U)
5	(1, 2, 1, 1, F)	F’ R U	(1, 2, 3, 1, U)
6	(1, 2, 1, 1, D)	F2	(1, 2, 3, 1, U)
7	(1, 1, 2, 1, F)	L’ U’	(1, 2, 3, 1, U)
8	(1, 1, 2, 1, L)	F	(1, 2, 3, 1, U)
9	(1, 3, 3, 2, U)	U	(1, 2, 3, 1, U)
10	(1, 3, 3, 2, R)	R’ F’	(1, 2, 3, 1, U)
11	(1, 3, 1, 2, R)	R F’	(1, 2, 3, 1, U)
12	(1, 3, 1, 2, D)	D’ F2	(1, 2, 3, 1, U)
13	(1, 1, 1, 2, D)	D F2	(1, 2, 3, 1, U)
14	(1, 1, 1, 2, L)	L’ F	(1, 2, 3, 1, U)
15	(1, 1, 3, 2, L)	L F	(1, 2, 3, 1, U)
16	(1, 1, 3, 2, U)	U’	(1, 2, 3, 1, U)
17	(1, 2, 3, 3, U)	U2	(1, 2, 3, 1, U)
18	(1, 2, 3, 3, B)	B’ R’ U	(1, 2, 3, 1, U)
19	(1, 3, 2, 3, R)	B’ D2 F2	(1, 2, 3, 1, U)
20	(1, 3, 2, 3, B)	R’ U	(1, 2, 3, 1, U)
21	(1, 2, 1, 3, D)	D2 F2	(1, 2, 3, 1, U)
22	(1, 2, 1, 3, B)	B R’ U	(1, 2, 3, 1, U)

23	(1, 1, 2, 3, L)	B D2 F2	(1, 2, 3, 1, U)
24	(1, 1, 2, 3, B)	L U’	(1, 2, 3, 1, U)

The following sub-section demonstrates the examples of the FSMs, made on the base of some of the solving sequences from this table.

3.2.5. Examples of the FSMs’ design

The planned application is a learning tool, therefore it will allow the user to make mistakes while solving the Cube. However, after two mistakes in row the application will terminate the current sequence and ask the user to start again. This is planned in order to keep the Cube from become scrambled too much from the half-solved state. The created FSMs reflect this tactic. The graph forms of the FSMs are created with the use of JFLAP software which is “a package of graphical tools which can be used as an aid in learning the basic concepts of Formal Languages and Automata Theory” (Rodger, 2011). Due to the input, allowed by this software, the alphabet as the whole is represented by the letter S instead of ∑.

The following examples of the FSMs are numbered after the list of the Cube’s states from the previous section.

3.2.4.1. Example of the zero transition FSM: 1. (1, 2, 3, 1, U)

The cubie is already in the correct position, so no extra move is required. The FSM in this case has only one state:

1. (1, 2, 3, 1, U).

The graph form will look as follows:

Figure 7 The FSM for the first starting state from the table (1, 2, 3, 1, U)

3.2.4.2. Example of the one main transition FSM: 3. (1, 3, 2, 1, R)

The cubie is moved into the correct position (1, 2, 3, 1, U) by the following sequence: F'. If any other turn of the front face was made, it still remains possible to get the cubie in the desired place by the additional turn of the front face. The turns of the most other faces won't influence the state of the Cube due to the fact that there are no solved cubies yet. Exception to this will be the turn of the right face, which will move the cubie that is being solved. In this case, the user will be asked to undo the move. Therefore, the states of the FSM will look like:

1. (1, 3, 2, 1, R);
2. (1, 2, 3, 1, U);
3. (1, 3, 3, 2, R);
4. (1, 3, 1, 2, R);
5. (1, 3, 2, 3, R);
6. (1, 2, 1, 1, D);
7. (1, 1, 2, 1, L);
8. (0, 0, 0, 0, 0).

The graph form will look as follows:

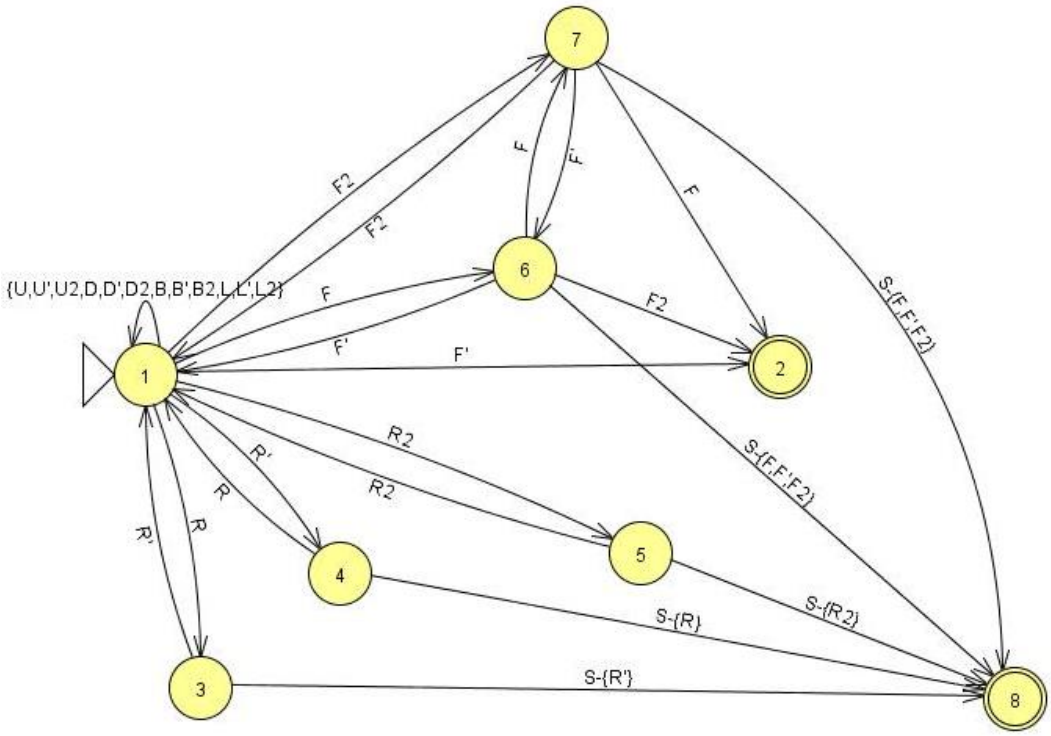

Figure 8 The FSM for the third starting state from the table (1, 3, 2, 1, R)

3.2.4.3. Example of the two main transition FSM: 4. (1, 3, 2, 1, F)

The cubie is moved into the correct position (1, 2, 3, 1, U) by the following sequence: R U. At the first move, if any other turn of the right face was made, it still remains possible to get the cubie in the desired place by additional turn of the right face. The turns of the most other faces

won’t influence the state of the Cube due to the fact that there are no solved cubies yet. Exception to this will be the turn of the front face, which will move the cubie that is being solved. In this case, the user will be asked to undo the move. At the second move, any turn of the right face will basically return the FSM to the first move. Any turn of the up face other than U will require additional turn of that face. Turns of the rest of the faces won’t influence the state of the cube in general. Therefore, the states of the FSM will look like:

1. (1, 3, 2, 1, F);
2. (1, 3, 3, 2, U);
3. (1, 2, 1, 1, F);
4. (1, 2, 3, 1, F);
5. (1, 1, 2, 1, F);
6. (1, 3, 2, 3, B);
7. (1, 3, 1, 2, D);
8. (0, 0, 0, 0, 0);
9. (1, 2, 3, 1, U);
10. (1, 1, 2, 1, U);
11. (1, 1, 3, 2, U).

The graph form will look as follows:

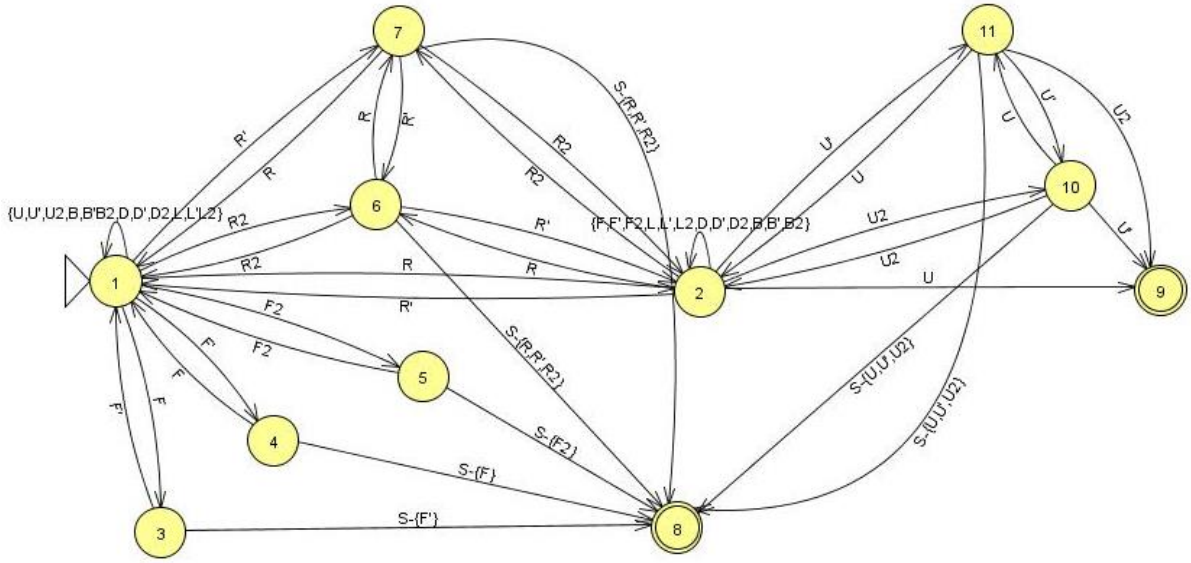

Figure 9 The FSM for the fourth starting state from the table (1, 3, 2, 1, F)

4. Development process

This chapter covers the development cycle for the planned application. As it was already mentioned in Chapter 1, the waterfall model was selected for the project. It is the oldest model of the software development process which is used since 1970 (Sommerville, 2015, p.47). This model divides the development process into a number of stages and each of those should be finished before the next one can be started. Those stages usually are:

- Requirements definition and analysis;
- System and software design;
- Implementation and unit testing;
- Integration and system testing;
- Operation and maintenance (Sommerville, 2015, p.47-48).

The sub-sections of this chapter are named after those stages: requirements definition, software design planning, implementation and testing. Each of the sub-sections will provide some background for its stage of the development process in addition to the description of that stage in the project.

4.1. Requirements definition

The requirements are "the descriptions of the services that a system should provide and the constraints of its operation" (Sommerville, 2015, p.102). They can be written in a natural language as well as in a formal way. The former are called user requirements and the latter – system requirements.

Requirements engineering is the first stage of the software development that mostly defines the further process since the view of the system is developed at that point (Sommerville, 2015, p.104). Requirements are often separated into functional and non-functional requirements and the same separation is made in this sub-section of the report.

4.1.1. Functional requirements

These requirements are the descriptions of the services that the system should provide as well as the descriptions of its behaviour in the different situations (Sommerville, 2015, p.105). They should be formulated as clearly as possible in order to avoid the miscommunication between the developers and the stakeholders.

The functionality, required from the project, can be written down in the following list of requirements:

- A 3D model of the Rubik's Cube should be used to demonstrate the solving moves;
- The 3D model should display the solved and unsolved cubies in two different colours and the selected cubie in a third colour;
- The 3D model should be able to demonstrate the moves of the solving sequence for the selected cubie;
- The user should be able to select a cubie by clicking it on the 3D model;
- The user should be able to select the step of the strategy they want to start from in case their Cube is partly solved;
- If the Cube is being solved from the beginning, instructions about terminology should be provided (corner, edge, cubie, Cube's colours);
- The program should not allow more than one mistake during the solving sequence. If the user makes two mistakes in the one solving sequence, it should be terminated and the program should go to the selection of the step of the strategy;
- After the selection of the step (or the start from the beginning) the program should demonstrate the solving sequences one by one until the Cube is solved or two mistakes are made during the one sequence;
- The functional for the extraction of the solving sequence from the FSM should be created and used for the demonstration of the steps of the sequence. Correctness of the moves, made by the user, should be checked with the use of the FSM;
- Correctness of the user's moves should be determined by checking the current position of the cubie that is being solved against its expected position according to the FSM;
- Notations for the sequence should be visible during the demonstration.

This list of requirements should be sufficient for the development of the first version of the software, which can later be refined according to the feedback from the users.

4.1.2. Non-functional requirements

These requirements are constraints of the offered functions and services, such as timing constraint (Sommerville, 2015, p.105). They specify the characteristics of the system as the whole.

For the project, a number of the non-functional requirements exists:

- The user should be able to use the application after reading the provided instructions;
- The model of the Cube should load under five seconds after the selection of the step;
- Application should not distract the user from learning;
- Application should be a Windows desktop program.

4.2. Software design planning

Before the software is implemented, its developer should produce the model of the software. Tools such as UML can be used for this. In case of this project, some parts of the software are modelled with the use of Finite State Machines (as described in Chapter 3). Those FSMs can be used alongside the other diagrams to create a complete model of the software.

This sub-section provides the description of the architecture of the application, its design with the use of UML diagrams as well as the examples of the planned UI (user interface).

4.2.1. Application architecture

The application architecture defines the overall structure of the system and the way it is organised (Sommerville, 2015, p.168). This structure should satisfy the existing requirements, functional as well as non-functional. While the architectural design is the unique process for each system, there are several general patterns which can be adapted into the design.

The planned application is supposed to work on a separated Windows machine independently of other copies. This makes it non-distributed, which excludes a number of possibly used patterns such as a client-server architecture. The application will combine several different elements, such as the implementation of FSMs, the Rubik's Cube's model and so on. Therefore the architecture should separate those parts. There are several design patterns that allow it, among them the layered architecture. It separates the different components into layers, where each layer only can use the services of the layer directly beneath it. From the point of view of the project, the following layers can be defined: user interface, Cube's model and core logic (FSMs).

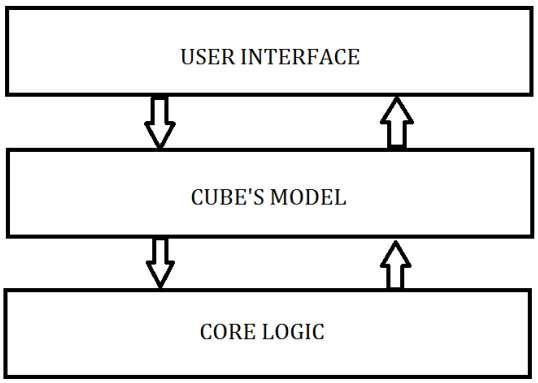

Figure 10 Application layers

Each layer has its own function. User interface allows the user to select the cubie they want to solve and passes this selection to the Cube's model (middle layer). Middle layer calls core logic layer and receives the solving sequence from it as well as operates the 3D model of the Cube in order to demonstrate this sequence.

4.2.2. Application design (UML)

In the beginning of the application design one needs to outline the business process model. Such model can generally help to organise the gathered requirements (Dennis, Wixom and Tegarden, 2010, p.159). This can be done with the use of several techniques, for example UML. Alternatively, a FSM that will describe the process can be built. In case of this project, however, the diagram form of such FSM would be rather complex, since all solving sequences needed to be included. Therefore, an UML diagram was used for the outline of the business process model:

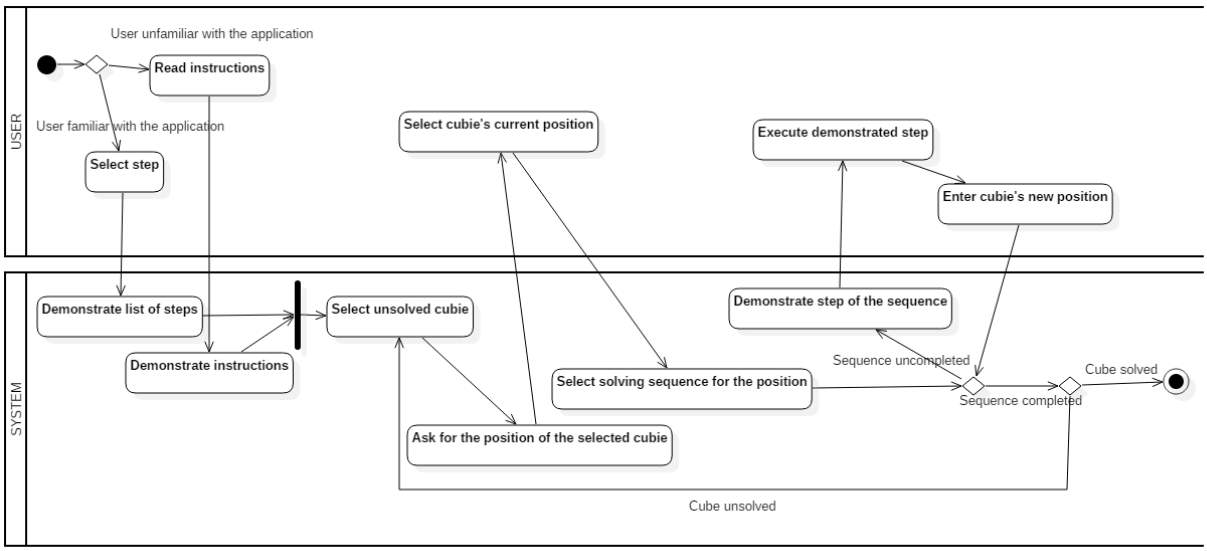

Figure 11 Business process UML model

The tool StarUML was used for the building of the diagram. Finite State Machines, demonstrated in the Chapter 3, are responsible for the demonstration of the sequence, therefore this model doesn't picture the check for the second mistake and termination of the sequence. In terms of the model it is assumed that the sequence is finished and a new cubie will be selected.

A use-case diagrams can be used to describe the system's functions from the point of view of the users (Dennis, Wixom and Tegarden, 2010, p.166). Depending on the amount of the functions, use cases can be fit into one or several diagrams. The next figure shows such diagram for the project:

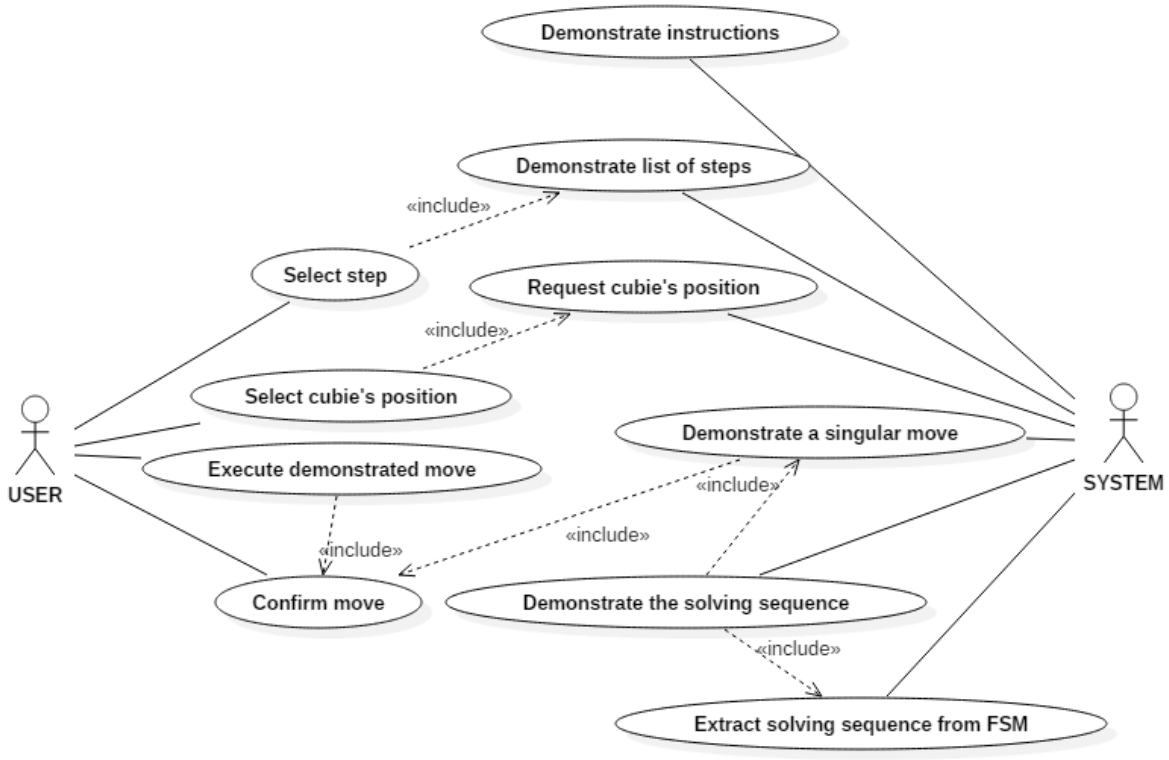

Figure 12 Use case UML diagram

For the development purposes, each use case has a text-based description. The next table demonstrates one of these descriptions on the example of "Extract solving sequence from FSM" use case.

Table 3 Use case text description

Name	Extract solving sequence from FSM
Goal	Get the shortest sequence of moves that will be accepted by the chosen FSM.
Initiator	System
Steps	1. Make a list of the possible accepted sequences.

	2. Calculate the length of each sequence from the list. 3. Select the one sequence with the smallest length.
Extensions	1. There are no accepted sequences except from the empty sequence. a. Inform that the cubie is already in the required position.

With the use cases describing the actions that need to be done to achieve the required functionality, class diagrams can be built. Their function will be to demonstrate the classes in the program and the relationships between them (Dennis, Wixom and Tegarden, 2010, p.213). The following figure demonstrates the planned contents and operations for the "Core logic" layer.

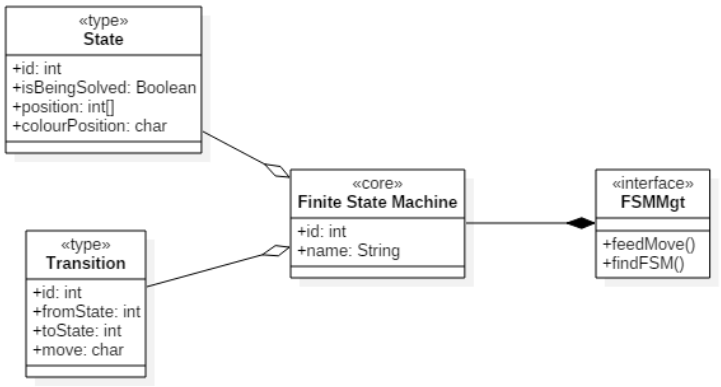

Figure 13 Class diagram for the "Core logic" layer

4.2.3. User Interface design

Generally, before the implementation can be started, it is necessary to create the mock-ups of the application screens. This will significantly simplify the implementation process in case there is more than one developer, which is usually the case with the commercial software development. In addition to those screens a general diagram is required. This diagram will show how the application is supposed to move from one screen to another.

Currently, there is a number of different tools that allow the creation of the required mock-up screens. A lot of them, however, are mostly oriented on the mobile applications and websites. This means that they can't be used for this project, which aims to create a Windows desktop application. One of the tools that support Windows screens is DesignerVista. This tool is easy to understand and use, so it was selected for the project.

The following diagram displays the transitions between the screens of the application.

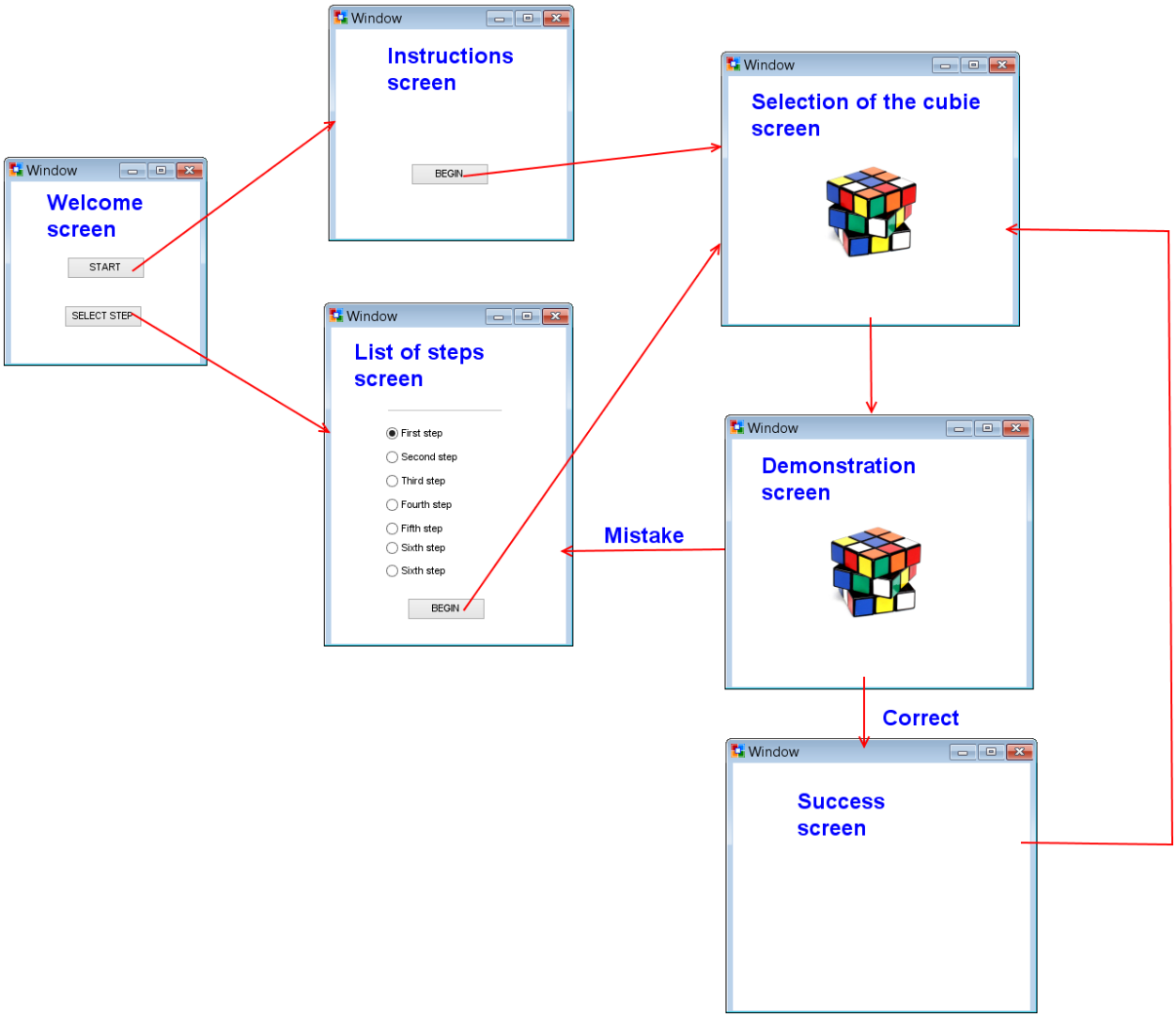

Figure 14 Diagram of the screens of the application and transitions between them

Since the application is potentially oriented on the children as its users, the general design of the application is adapted so that it would be easier to use for the children. There are several age groups with different understanding of the elements of the interfaces (Hourcade, 2007, p.286). This understanding grows with age, so an application, designed for the earliest age group that can potentially be interested in the Rubik's Cube (2-7 years old), will also be understandable for the older children. For that age group (called preoperational children), for example, one should avoid the use of the hierarchies in the interface design. The reason for it is the fact that these children can only concentrate on the one characteristic of an object at a time (Hourcade, 2007, p.286). That is why the application screens should be as simple as possible.

Now, when the general diagram is developed, the individual screens can be made according to it. The following figures demonstrate the planned design of the separate application screens.

Figure 15 Welcome screen

Since the application is created for the individual learning, it doesn't connect to the other clients or any common server. This means that there is no need for the login feature on the welcome screen.

The next figure demonstrates the instructions screen.

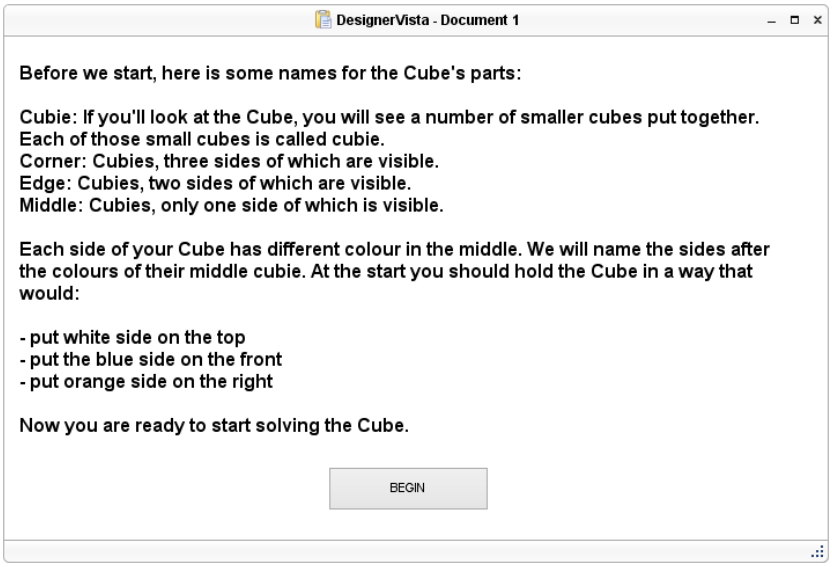

Figure 16 Instructions screen

This screen provides general instructions that are necessary in order to understand more specific instructions for the solutions of the separate cubies. If the Cube is partly solved, it will be assumed that the user is already familiar with the terminology and this screen won't appear. Instead, the selection of the specific step will be offered:

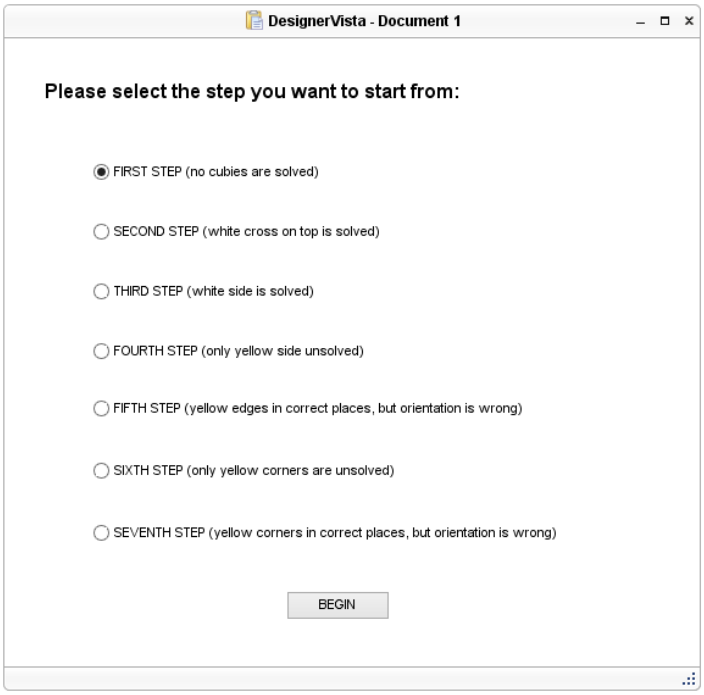

Figure 17 List of steps screen

Screens from both figures 13 and 14 will lead to the same screen, where the user will be asked to select the current position of the cubie they will be solving at this point:

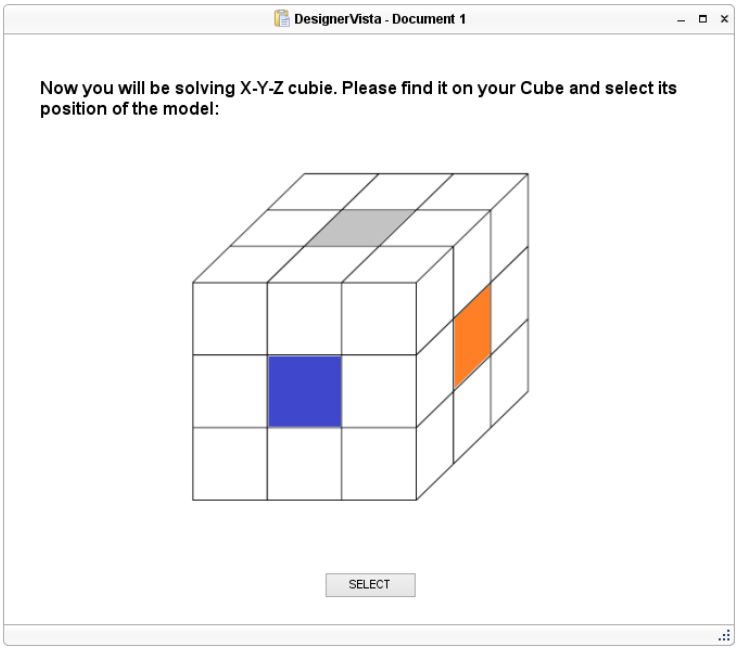

Figure 18 Cubie selection screen

The "X-Y-Z" notation will describe the colour of the cubie's sides, for example Yellow-Blue-Orange. When the edge cubie will be solved, the third colour will not appear and the description will look like Blue-White. The user will be able to turn the model in case the cubie won't be on the visible side of the Cube. After selection, the user will be taken to the next screen with the demonstration of the required solving sequence:

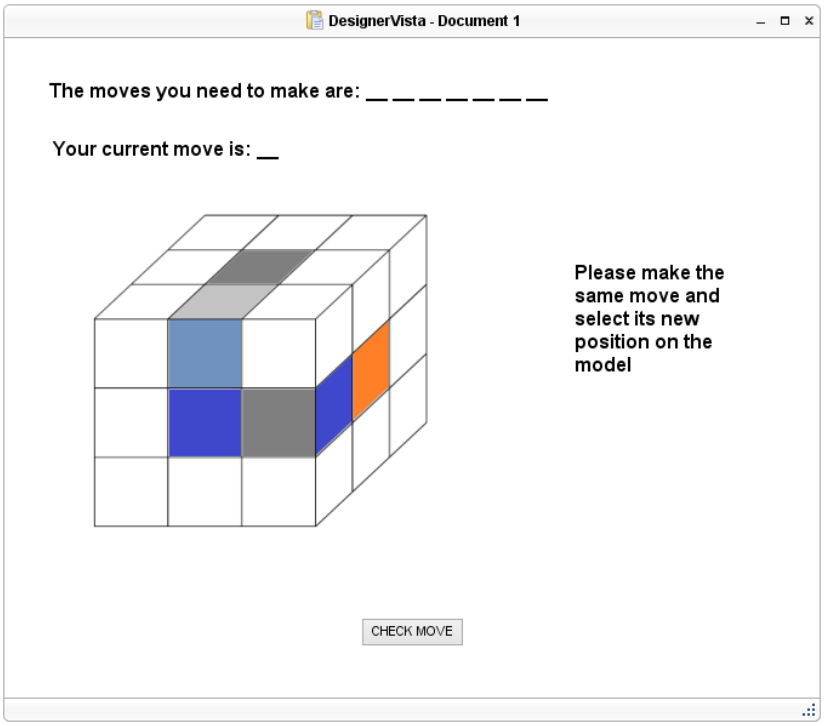

Figure 19 Demonstration screen

On the model on this screen two cubies are highlighted: the desired end position of the cubie and its current position. The first one uses a lighter versions of the Cube's colours. The user will be shown the moves from the sequences and will be asked to repeat them. After each move the application will require the confirmation of the move. For that user will need to select the position of the cubie after the move was made. If the move was right, the application will stay on the same screen and will demonstrate the next move from the sequence. If the move was wrong and it was the first mistake, the application will warn the user and offer the move that is required to rectify that mistake. After the second mistake the user will be taken to the screen with the list of steps in order to restart the solution. If the sequence was completed successfully, the user will be taken to the success screen:

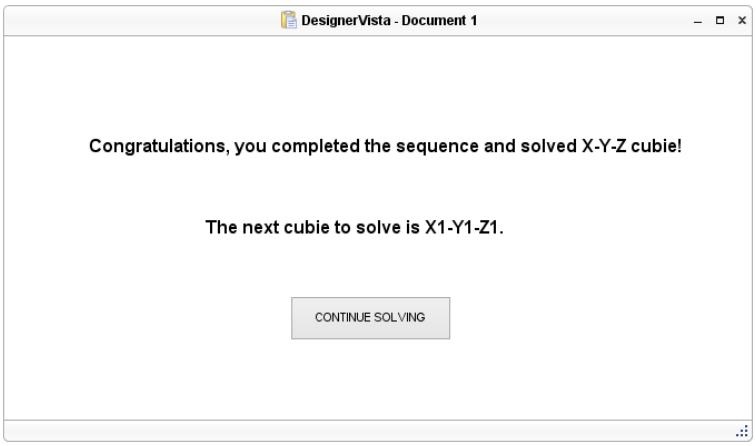

Figure 20 Success screen

From there user will again be taken to the selection of the cubie screen and the cycle will continue until the puzzle is solved.

4.3. Implementation

At this stage an actual software is developed based on the specifications that were created earlier. This sub-section provides the description of the used tools, describes the implementation process and demonstrates the examples of the application code.

4.3.1. Implementation tools

The application is supposed to be a Windows desktop program, which uses a 3D model of the Rubik's Cube to demonstrate the solving moves. This requirement influences the selection of the programming language. In order to create a 3D model, additional tools are required. One of the widely used tools for the creation of the graphics is OpenGL. It is an "open-standard cross-platform and cross-language 3D graphics API" (Guha, 2015, p.9) that was first released in 1992. OpenGL can be used in many programming languages, but all those bounds are based on C/C++ (OpenGL, 2016). C++ is an object-oriented general purpose language that is considered very effective. This means that it can be used as the programming language of the project.

There is a two often used integrated development environments (IDEs) that support C++ programming: Visual Studio and Dev-C++. Former is developed by Microsoft and latter is a GNU General Public License software. OpenGL integrates easier with Visual Studio, so this IDE was selected for the project.

4.3.2. Implementation process

In order to help with the organisation of the code so that it would be readable, three groups of classes were created. Each of those groups represents the one layer of the application. The first layer, "Core logic" is built around the implementation of Finite State Machines. There are several possible ways of implementation of a FSM. One of them is the design pattern called "State pattern" (Gamma *et al.,* 1995, p.305). This pattern contains an abstract class that represents the state. Subclasses of that class are made in order to create the specific states. Another class is made to maintain the current state.

An alternative to this pattern is the creation of a table that maps inputs to state transitions (Gamma *et al.*, 1995, p.308). This methods makes it easier to modify the data but simultaneously makes the transitions less obvious. In case of the project, this alternative method is preferable due to the fact that there are many FSMs with the similar structure in the code. Because of this they would require a lot of separate classes and the code would become even more complicated than with the use of the tables. The description of the implementation of the FSM as a table can be found in the next sub-section.

The middle layer contains the method, which takes a FSM as an input and extracts the solving sequence from it. All FSMs, created for the project, share the same structure and extraction algorithm is based on that structure. In any FSM there is only one accepting state that is not equal (0, 0, 0, 0, 0). This state is the one in which the FSM should end if presented with an acceptable solving sequence. Therefore, the method needs to find the shortest way to that state. In order to do this, it will do the following:

1. The FSM will be given all characters from its alphabet (∑: {U, D, L, R, F, B, U', D', L', R', F', B', U2, D2, L2, R2, F2, B2}) as an input. The states, in which the FSM will be brought will be saved.
2. The list of states will be analysed and the records, where the state equals (0, 0, 0, 0, 0) or is the starting state, will be removed. If no state is an accepting state that is not equal (0, 0, 0, 0, 0), the method will go to the next step.
3. The next step is similar to the first one, but now the FSM will be given a two-character input: the first character will be taken from the saved list and the second one will be taken from the alphabet, so that any possible combination could be checked. In general the procedure is close to the building of a tree. The list of states will be rewritten.
4. The list of states will be analysed same as during the second step. If the required accepting state won't be found, method will move onto the next step, where a three-character input will be analysed. This cycle will continue until the required accepting state will be found and the sequence that led to it will be returned as the result.

The top layer contains the user interface. The application was developed as the Win32 application and the corresponding language tools were used for the creation of the UI.

4.3.3. Code examples

This sub-chapter contains the examples of the code with commentaries. As the example the FSM, described in the sub-section 3.2.4.2 was taken. An example, which can be found at http://www.gedan.net/2008/09/08/finite-state-machine-matrix-style-c-implementation/ was used as a base for this code.

For the implementation this FSM was brought to the table form. Firstly, the states and transitions were defined:

```
typedef enum {
	STATE1321R,
	STATE1231U,
	STATE1332R,
	STATE1312R,
	STATE1323R,
	STATE1211D,
	STATE1121L,
	STATE00000
} state;

typedef enum {
	U, D, L, R, F, B, UU, DD, LL, RR, FF, BB, U2, D2, L2, R2, F2, B2
} event;
```

The transitions (events) and states were named according to the notations, defined in the Chapter 3. The only change was made to the notations like “U’” which were changed into “UU” and so on. The structure of the FSM was defined on their basis:

```
typedef struct {
	event eventInput;
	state nextState;
	bool isFinal;
}  stateElement;
```

The structure consists out of three elements: the input to the FSM (eventInput), the state it should go in case of that input (nextState) and the variable that shows if the state is accepting or not (isFinal). Then the FSM itself was defined as an 8x18 matrix. For example, the first state was defined as follows:

```
{
{FSMtable::U,FSMtable::STATE1321R,false },{FSMtable::D,FSMtable::STATE1321R,false },{FSMtable::L,FSMtable::STATE1321R,false },
{FSMtable::R,FSMtable::STATE1332R,false },{FSMtable::F,FSMtable::STATE1211D,false },{FSMtable::B,FSMtable::STATE1321R,false },
{FSMtable::UU,FSMtable::STATE1321R,false },{FSMtable::DD,FSMtable::STATE1321R,false },{FSMtable::LL,FSMtable::STATE1321R,false },
{FSMtable::RR,FSMtable::STATE1312R,false },{FSMtable::FF,FSMtable::STATE1231U,false },{FSMtable::BB,FSMtable::STATE1321R,false },
{FSMtable::U2,FSMtable::STATE1321R,false },{FSMtable::D2,FSMtable::STATE1321R,false },{FSMtable::L2,FSMtable::STATE1321R,false },
{FSMtable::R2,FSMtable::STATE1323R,false },{FSMtable::F2,FSMtable::STATE1121L,false },{FSMtable::B2,FSMtable::STATE1321R,false }
},
```

For the each one of the possible eighteen inputs the state, in which the FSM should go, is defined. The rest of the states of the FSM are defined in the same way. In order to work with this FSM, a method that would read it is needed.

```
bool readState(FSMtable::event e, FSMtable::stateElement fsm[8][18])
{
	FSMtable::stateElement findState = fsm[currentState][e];
	currentState = findState.nextState;
	return fsm[currentState][e].isFinal;
}
```

This method takes the FSM and the input and finds the state that FSM will go to after that input. It returns the information about whenever that state is accepting or not, which allows to check the acceptance of a given sequence by the FSM.

4.4. Testing

When the code of the application is written, it should be tested. There are several types of testing: the unit testing, the component testing and the system testing (Sommerville, 2015, p.232). The unit testing checks the functionality of individual objects, such as classes, while the component testing focuses on the ability of interfaces to provide the necessary functions. The system testing checks the software as a whole. Those types of testing are supposed to be done one after another, starting with the unit testing. The following sub-section provides the tests, designed for the testing of the individual objects and the sub-section after it demonstrates the results of those tests.

4.4.1. Testing scenarios

The application structure is planned to be layered, so the tests will be grouped according to these layers.

The bottom layer, called “Core logic”, contains the FSMs and the functions that allow the middle layer to work with them. In order to check the work of this layer, the following should be tested:

1. If no suitable FSMs exist then the method *findFSM(cubie, position)* returns 0;
2. If there is a suitable FSM then the method *findFSM(cubie, position)* returns id of this FSM;
3. The method *feedMove(idFSM, move)* should return a state, in which the FSM ends after the move;
4. The method *feedMove(idFSM, move)* should accept an array as its second parameter and return a state, in which the FSM ends after all moves in that array;

The middle layer, called “Cube’s model”, contains firstly the model of the Cube and secondly the method, extracting the solving sequence from the FSM. For this method, the following should be tested:

5. The shortest solving sequence is returned;
6. The returned solving sequence is correct for the selected FSM;

The top layer, called “User Interface”, contains the code of the user interface as well as the methods that allow the user to select the step of the strategy and, after that, the position of the cubie that is currently being solved. The cubies, belonging to each step, and their order are encoded and can’t be changed by the user. The following will need to be tested:

7. The user interface has the same look and structure as in the mock-ups from the sub-chapter 4.2.3;
8. The solving sequence, produced for the selected cubie is correct.

The next sub-chapter will demonstrate those tests, done with the specific data, such as the FSM for the selected cubie (Blue-White) in the selected position.

4.4.2. Testing results

The testing was done of the FSM, described in the sub-section 3.2.4.2. The solving sequence of that FSM is F’. According to the third test, the code should show the current state of the FSM. The method, described in the sub-section 4.3.3 allows this:

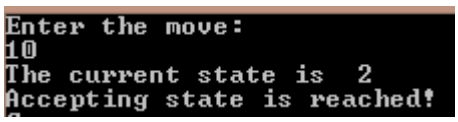

Figure 21 Testing the FSM's correctness

The entered move is the number of the move F’ (starting with 0) in the array of all possible moves (FSM’s alphabet).

5. Evaluation and conclusion

This chapter contains the evaluation of the application from two points of view. The first is the general evaluation of the application's effectiveness and the second one is concentrated on the use of Finite State Machines in the development of the application. In addition, the chapter also contains the conclusion of the work on the project.

5.1. Application evaluation

The goal of the project was to create an application that would be a learning tool for the Rubik's Cube. One of the existing solving strategies was selected and divided into the separate solving sequences in order to demonstrate them in the application.

The demonstration of the actual moves combined with the display of the sequence itself makes the process of studying and remembering the sequences easier than if the only one method was used. However, since the user might not learn them after the first demonstration, the option to select a required step was introduced in the application. This option allows the user to skip the steps they are familiar with and see the demonstration of the more complicated sequences whenever they want to. This in turn increases the learning value of the application.

As for the fitness for the use by children, the application requires an additional work. This is due to the fact that the language of the instructions, presented in the application, might be too complicated for the children under 7. Therefore, an additional UI must be created in order to make an application accessible to the small children.

Compared to the other existing applications, for example at https://rubiks-cube-solver.com/, this program does not offer a complete solution to the scrambled Cube. Instead it demonstrates the solution of the selected cubie, which is more helpful for the learning of the Cube's solutions. The program also doesn't allow the user to enter the current state of their Cube at the beginning (which the provided online application does). This feature would enhance the program, so it might be added in the future.

5.2. Evaluation of the use of FSMs in the development of the application

The Finite State Machines were used in the application development to store and organise the logic of the application. More specifically, they represented the solving sequences for the

separate positions of the different cubies, one FSM per sequence. This required more preparation work to be done before the implementation phase could be started, but this was balanced by reducing the time needed for the implementation phase due to the unification of the FSMs code. As the result, not only was the implementation phase simplified, but also the chances of making mistakes in the code were lessened because the logic was already organised in the FSMs.

Since the solving sequences (logic) of the application were put in the code in form of FSMs, no additional storage for the data (such as a database) was required. This further simplified the structure of the implementation phase and made the code of the application more compact. However, the creation of the tool to extract the optimal solving sequence from the FSMs was required because of that. The recognition strategy was adjusted to the structure of the created FSMs, which simplified its creation. This adaptation of the tool was made because FSMs can be vastly different for the different application areas and, therefore, one common tool would require too much time and effort that would be practical for the task.

The use of the FSMs also allows to add the additional strategies without disrupting the application as the whole, which will be good for its maintenance in the future.

5.3 Conclusion of the work

Several things were done during the work on the project. Firstly, the solving strategy for the Rubik's Cube was selected and analysed. During this analysis the separate solving sequences were extracted. Then a number of FSMs was produced based on the extracted sequences. Those FSMs were laid in the base of the application. Therefore, three out of four objectives were completed. The fourth objective, analysis of the effectiveness of FSMs as a basis of the software, was done in the sub-chapter 5.2.

To summarise the results, the usage of the Finite State Machines in the development of the software generally proved to be successful, but required an exact planning before the start of the implementation phase. Therefore, their use can potentially hinder the development process in case the stakeholders don't have specifications of the project ready yet. Most of the time, however, at least the base functionality can be written down and used for the building of FSMs. After this, due to the fact that FSMs organise the structure of the software, additional functionality can be added in the same way. So it can be said that the starting hypothesis was mostly confirmed and the Finite State Machines proved themselves as a functional tool for the software creation.

The original plan was to make the learning tool that would be fit for the children, but during the work on the project it became obvious that an additional research on the organisation of the UI would be required, so that adaptation was moved into the future work.

6. Future work

This chapter describes the work that needs to be done in order to complete the application and to ensure that it can be used by the intended age group. The chapter consists of the two sub-chapters: one about application itself and another about the use of the Finite State Machines in the application development.

6.1. Application development

During the application development the main elements, such as the Finite State Machines and the tool for working with them were created. However, in order to make the application fully functional, all steps of the strategy need to be implemented. The application also can benefit from the addition of the descriptions of the strategy's steps, which will show the user the whole picture rather than just leading them from the one solved cubie to another.

To expand the application and make it a better learning tool, an exercises can be added to it. The goal of such exercises will be either to recreate the required solving sequence from the memory (harder option) or to follow the sequence that will be demonstrated without breaks between the moves (easier option). These exercises will allow the user to refresh the learned sequences in their memory. Also the application would benefit from the option that would allow the user to enter the current state of their Cube before start.

While the UI of the created application was structured with the children needs in mind, it still requires some refinement. For example, current instruction screen uses some terms that may be not easily understood by children. Therefore, an additional research is required. The goal of that research will be to put together a list of ways of making the information accessible to the children and to create and test an additional UI based on that list. Current UI may still serve for the teenage or the adult users of the application and it will be possible to switch the children's UI on if needed.

6.2. Use of FSMs in software development

During the work on the project the Finite State machines were used in the development and the result was evaluated in the previous chapter. This evaluation shows that the FSMs allow the developer to keep the logic of the application in a unified format and in the one place. This suggests that the method, used for the extraction of the logic from the FSMs could be used in an

application, created for the other business areas. A development of the exact way of its adaptation and use could be done in future.

7. References

Attwood, T. (2006) *The complete guide to Asperger's syndrome*. London. Jessica Kingsley.

Berndtsson, M., Hansson, J., Olsson, B. and Lundell, B. (2008) *Thesis projects: a guide for students in computer science and information systems.* London. Springer.

Davis, N. (2014) *How Ernö Rubik Created the Rubik's Cube*. Available at: http://mentalfloss.com/article/58162/how-erno-rubik-created-rubiks-cube (accessed: 30 March 2016).

Demaine, E.D., Demaine, M.L., Eisenstat, S., Lubiw, A. and Winslow, A. (2011) 'Algorithms for solving Rubik's cubes', *Algorithms–ESA 2011.* Springer Berlin Heidelberg. pp. 689-700.

Dennis, A., Wixom B.H. and Tegarden D. (2010) *Systems Analysis and Design with UML. An object-oriented approach.* 3rd edn. John Wiley and Sons.

El-Sourani, N., Hauke, S. and Borschbach, M. (2010) 'An evolutionary approach for solving the Rubik's cube incorporating exact methods'. *Applications of Evolutionary Computation*. Springer Berlin Heidelberg. pp. 80-89.

Gamma, E., Helm, R., Johnson, R., Vlissides, J. (1995) *Design patterns: elements of reusable object-oriented software*. Addison-Wesley.

Goddard, W. (2008) *Introducing the Theory of Computation*. Jones & Bartlett Learning.

Guha, S. (2015) *Computer Graphics Through OpenGL: From Theory to Experiments*. CRC Press.

Hong, P., Turk, M. and Huang, T.S. (2000) 'Gesture modeling and recognition using finite state machines' *Proceedings. Fourth IEEE international conference on Automatic face and gesture recognition.* pp. 410-415.

Houghton Mifflin Company. *The American Heritage® Science Dictionary.* Available at: http://www.dictionary.com/browse/finite-state-machine (accessed: 29 March 2016).

Hourcade, J. P. (2007) 'Interaction design and children'. *Foundations and Trends in Human-Computer Interaction*, 1(4), pp. 277–392.

Kendall, G., Parkes, A. and Spoerer, K. (2008) 'A Survey of NP-Complete Puzzles' *International Computer Games Association Journal*, 31(1), pp. 13–34.

Korf, R.E. (1997) ‘Finding optimal solutions to Rubik's cube using pattern databases’ *AAAI/IAAI.* pp. 700-705.Lawson, M.V. (2003) *Finite Automata*. CRC Press.

Lawson, M.V. (2003) *Finite Automata*. CRC Press.

Monga, A. and Singh, B. (2012) ‘Finite State Machine based Vending Machine Controller with Auto-Billing Features’ *International Journal of VLSI design and Communication Systems*, 3(2), pp. 19-27.

Neapolitan R.E. (2015) *Foundations of algorithms. 5th edn*. Burlington. Jones and Bartlett Learning.

OpenGL (2016) *Getting Started*. Available at: https://www.opengl.org/wiki/Getting_Started (Accessed: 31 July 2016).

Qureshi, N.S., Mushtaq, H., Aslam, M.S., Ahsan, M., Ali, M. and Atta, M.A. (2012) ‘Computing Game Design with Automata Theory’ *International Journal of Multidisciplinary Sciences and Engineering*, *3*(5), pp. 13-21.

Rivest, R.L. and Schapire, R.E. (1987) ‘Diversity-based inference of finite automata’ *IEEE 28th Annual Symposium on Foundations of Computer Science*, pp. 78-87.

Rokicki, T., Kociemba, H., Davidson, M. and Dethridge, J. (2013) ‘The Diameter of the Rubik's Cube Group Is Twenty’ *SIAM Journal on Discrete Mathematics*, *27*(2), pp. 1082-1105.

Rodger S.H. (2011) *What Is JFLAP?* Available at: http://www.jflap.org/ (Accessed: 19 June 2016).

Skorkin, A. (2011) *Why Developers Never Use State Machines*. Available at: http://www.skorks.com/2011/09/why-developers-never-use-state-machines/ (accessed: 4 May 2016).

Slocum, J., Singmaster, D., Huang, W.H., Gebhardt, D. and Hellings, G. (2009) *The Cube: The Ultimate Guide to the World’s Bestselling Puzzle — Secrets, Stories, Solutions.* Black Dog & Leventhal Publishers.

Sommerville, I. (2015) *Software engineering. Tenth edition.* Pearson: Addison-Wesley.

Taylor, L.A. and Korf, R.E. (1993) ‘Pruning Duplicate Nodes in Depth-First Search’. *Proceedings of the eleventh national conference on Artificial intelligence.* pp. 756-761.

van Bergen, W. (2011). *Why developers should be force-fed state machines*. Available at: https://engineering.shopify.com/17488160-why-developers-should-be-force-fed-state-machines (accessed: 4 May 2016).

Vega Society. *Rubik's Cube – Spatial Intelligence – Brain Training*. Available at: http://www.vegasociety.com/brain/rubiks.html (accessed 31 March 2016).

Wagner F., Schmuki R., Wagner T. and Wolstenholme P. (2006) *Modeling Software with Finite State Machines: A Practical Approach.* CRC Press.

Wood D. (1987) *Theory of computation*. John Wiley and Sons.

World Cube Association. *Records*. Available at: https://www.worldcubeassociation.org/results/regions.php (accessed 31 March 2016).

Wright D.R. (2005). CSC216 (Summer 2005). Finite State Machines. Available at: http://www4.ncsu.edu/~drwrigh3/docs/courses/csc216/fsm-notes.pdf (accessed 25 July 2016).

Zuzak, I., Budiselic, I. and Delac, G. (2011) 'A finite-state machine approach for modeling and analyzing restful systems' *Journal of Web Engineering*, 10(4), pp. 353-390.

Words count

The number of words, excluding references, appendices and titles for figures and tables: 14 967.

Appendix 1. FSM diagrams and description.

6. (1, 2, 1, 1, D). The cubie is moved into the correct position (1, 2, 3, 1, U) by the following sequence: F2. If any other turn of the front face was made, it still will be possible to get the cubie in the desired place by additional turn of the front face. The turns of the most other faces won't influence the state of the Cube due to the fact that there is no solved cubies yet. Exception to this will be the turn of the down face, which will move the cubie that is being solved. In this case, user will be asked to undo the move. Therefore, states of the FSM will look like:

1. (1, 2, 1, 1, D);
2. (1, 2, 3, 1, U);
3. (1, 3, 1, 2, D);
4. (1, 1, 1, 2, D);
5. (1, 2, 1, 3, D);
6. (1, 1, 2, 1, L);
7. (1, 3, 2, 1, R);
8. (0, 0, 0, 0, 0).

The graph form will look as follows:

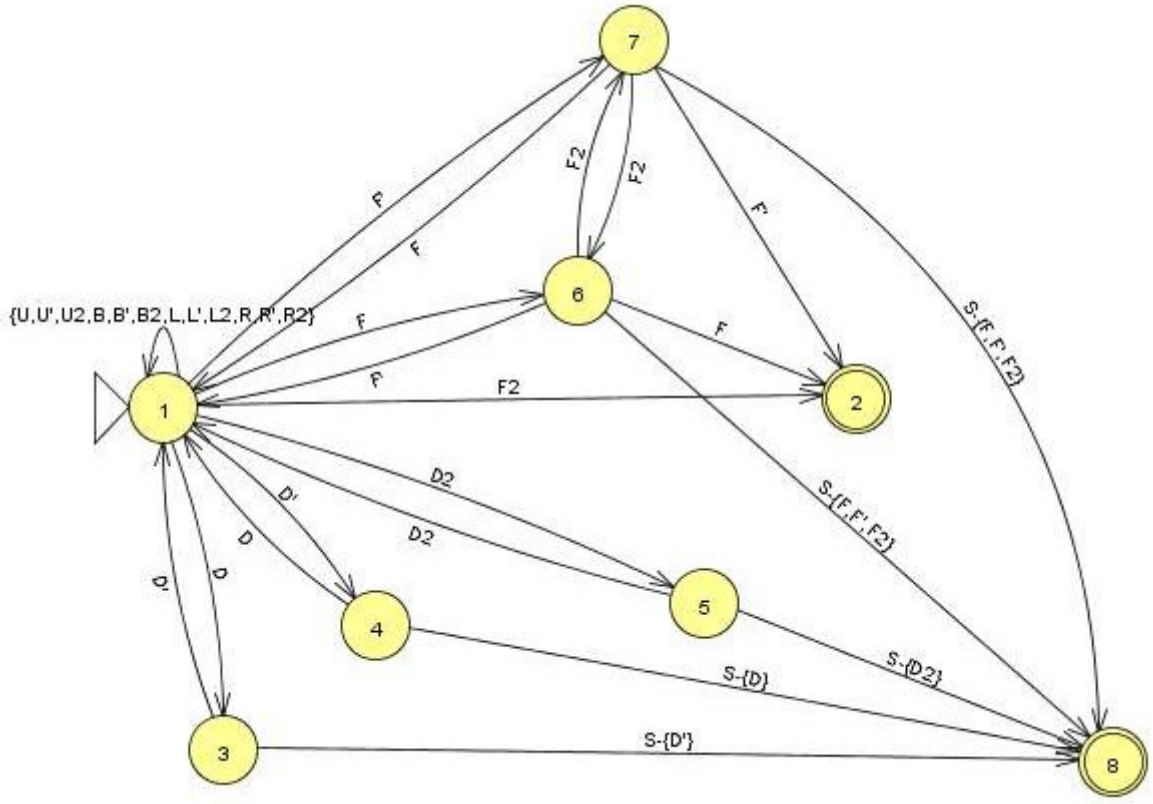

Figure 22 FSM for the sixth starting state from the list (1, 2, 1, 1, D)

8. (1, 1, 2, 1, L). The cubie is moved into the correct position (1, 2, 3, 1, U) by the following sequence: F. If any other turn of the front face was made, it still will be possible to get the cubie

in the desired place by additional turn of the front face. The turns of the most other faces won't influence the state of the Cube due to the fact that there is no solved cubies yet. Exception to this will be the turn of the left face, which will move the cubie that is being solved. In this case, user will be asked to undo the move. Therefore, states of the FSM will look like:

1. (1, 1, 2, 1, L);
2. (1, 2, 3, 1, U);
3. (1, 1, 1, 2, L);
4. (1, 1, 3, 2, L);
5. (1, 1, 2, 3, L);
6. (1, 2, 1, 1, D);
7. (1, 3, 2, 1, R);
8. (0, 0, 0, 0, 0).

The graph form will look as follows:

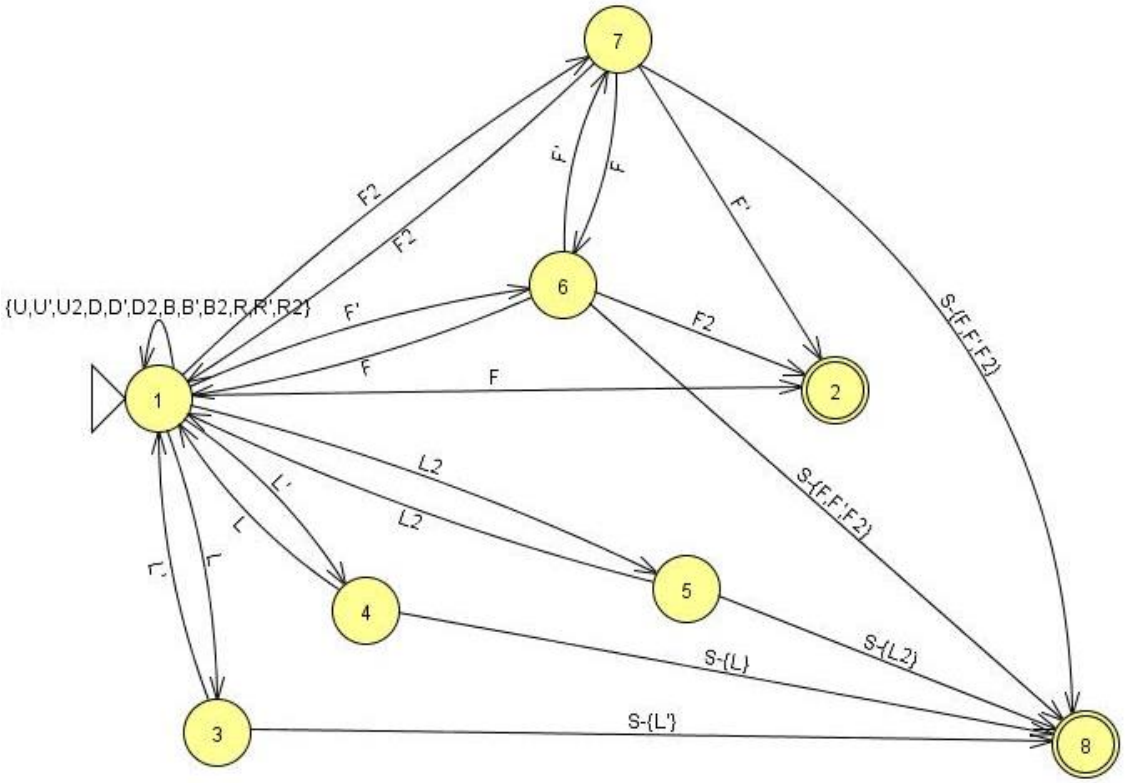

Figure 23 FSM for the eighth starting state from the list (1, 1, 2, 1, L)

9. (1, 3, 3, 2, U). The cubie is moved into the correct position (1, 2, 3, 1, U) by the following sequence: U. If any other turn of the up face was made, it still will be possible to get the cubie in the desired place by additional turn of the up face. The turns of the most other faces won't influence the state of the Cube due to the fact that there is no solved cubies yet. Exception to this will be the turn of the right face, which will move the cubie that is being solved. In this case, user will be asked to undo the move. Therefore, states of the FSM will look like:

1. (1, 1, 3, 2, U);
2. (1, 2, 3, 1, U);

3. (1, 3, 2, 3, B);
4. (1, 3, 2, 1, F);
5. (1, 3, 1, 2, D);
6. (1, 2, 3, 3, U);
7. (1, 1, 3, 2, U);
8. (0, 0, 0, 0, 0).

The graph form will look as follows:

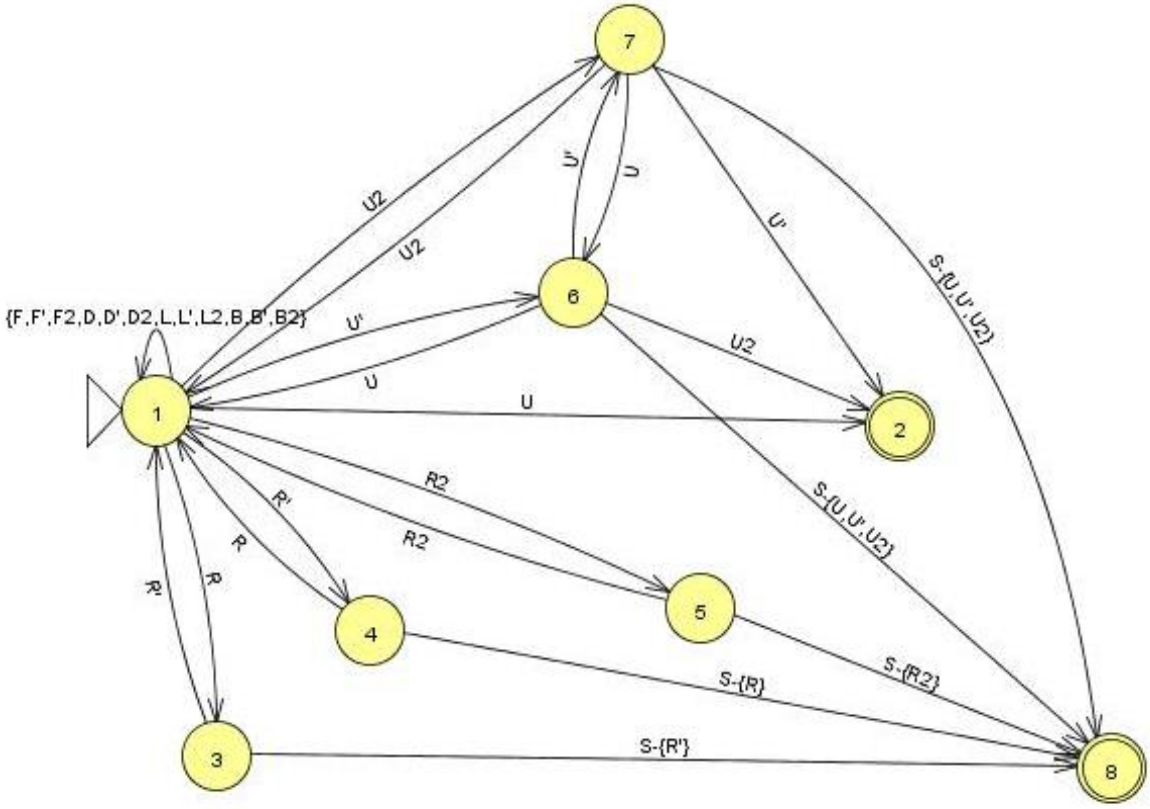

Figure 24 FSM for the ninth starting state from the list (1, 3, 3, 2, U)

12. (1, 3, 1, 2, D). The cubie is moved into the correct position (1, 2, 3, 1, U) by the following sequence: D' F2. At the first move, if any other turn of the down face was made, it still will be possible to get the cubie in the desired place by additional turn of the down face. The turns of the most other faces won't influence the state of the Cube due to the fact that there is no solved cubies yet. Exception to this will be the turn of the right face, which will move the cubie that is being solved. In this case, user will be asked to undo the move. At the second move, any turn of the down face will basically return the FSM to the first move. Any turn of the front face other than F2 will require additional turn of this face. Turns of the rest of the faces won't influence the state of the cube in general. Therefore, states of the FSM will look like:

12. (1, 3, 1, 2, D);
13. (1, 2, 1, 1, D);
14. (1, 3, 2, 1, F);
15. (1, 3, 2, 3, B);

16. (1, 3, 3, 2, U);
17. (1, 2, 1, 3, D);
18. (1, 1, 1, 2, D);
19. (0, 0, 0, 0, 0);
20. (1, 2, 3, 1, U);
21. (1, 1, 2, 1, L);
22. (1, 3, 2, 1, R).

The graph form will look as follows:

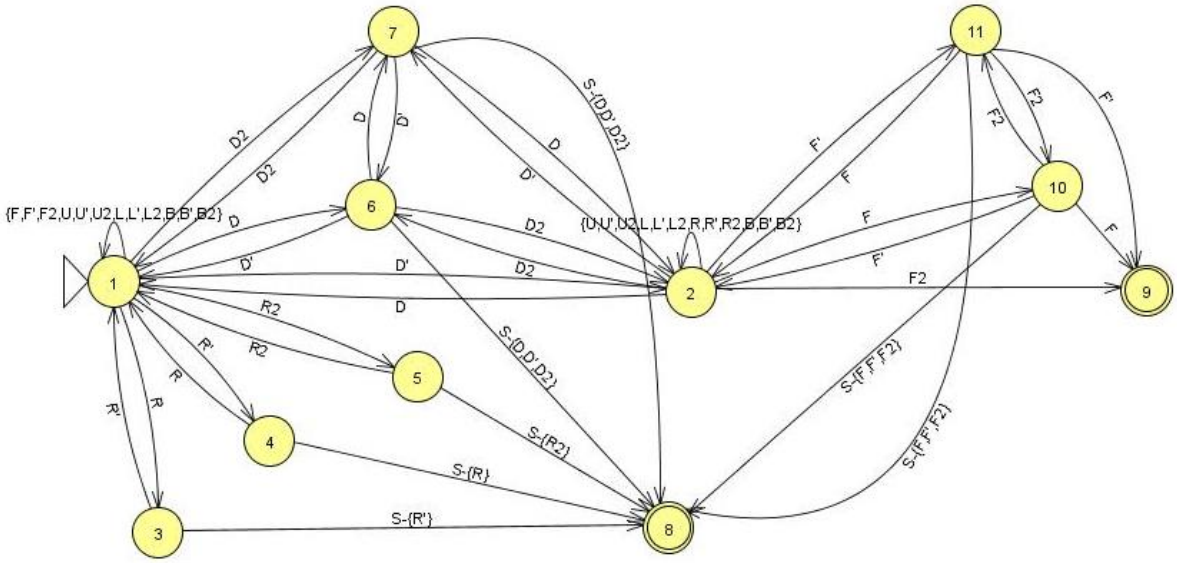

Figure 25 FSM for the twelfth starting state from the list (1, 3, 1, 2, D)

13. (1, 1, 1, 2, D). The cubie is moved into the correct position (1, 2, 3, 1, U) by the following sequence: D F2. At the first move, if any other turn of the down face was made, it still will be possible to get the cubie in the desired place by additional turn of the down face. The turns of the most other faces won't influence the state of the Cube due to the fact that there is no solved cubies yet. Exception to this will be the turn of the left face, which will move the cubie that is being solved. In this case, user will be asked to undo the move. At the second move, any turn of the down face will basically return the FSM to the first move. Any turn of the front face other than F2 will require additional turn of this face. Turns of the rest of the faces won't influence the state of the cube in general. Therefore, states of the FSM will look like:

1. (1, 1, 1, 2, D);
2. (1, 2, 1, 1, D);
3. (1, 1, 2, 3, B);
4. (1, 1, 2, 1, F);
5. (1, 1, 3, 2, U);
6. (1, 3, 1, 2, D);
7. (1, 2, 1, 3, D);
8. (0, 0, 0, 0, 0);
9. (1, 2, 3, 1, U);
10. (1, 1, 2, 1, L);

11. (1, 3, 2, 1, R).

The graph form will look as follows:

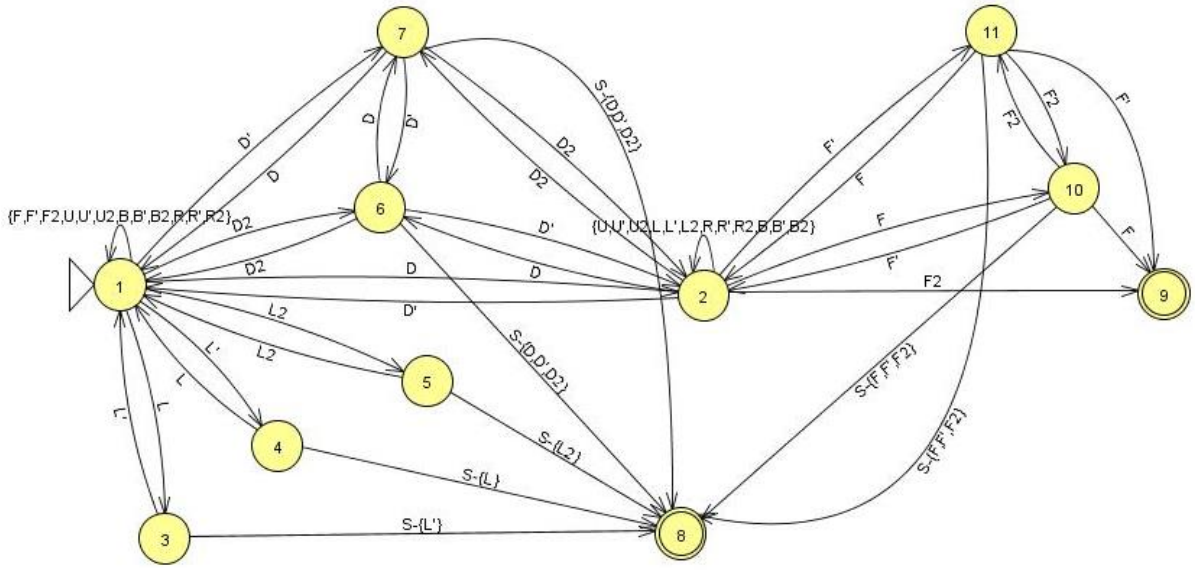

Figure 26 FSM for the thirteenth starting state from the list (1, 1, 1, 2, D)

16. (1, 1, 3, 2, U). The cubie is moved into the correct position (1, 2, 3, 1, U) by the following sequence: U'. If any other turn of the up face was made, it still will be possible to get the cubie in the desired place by additional turn of the up face. The turns of the most other faces won't influence the state of the Cube due to the fact that there is no solved cubies yet. Exception to this will be the turn of the left face, which will move the cubie that is being solved. In this case, user will be asked to undo the move. Therefore, states of the FSM will look like:

1. (1, 1, 3, 2, U);
2. (1, 2, 3, 1, U);
3. (1, 1, 2, 1, F);
4. (1, 1, 2, 3, B);
5. (1, 1, 1, 2, D);
6. (1, 2, 3, 3, U);
7. (1, 3, 3, 2, U);
8. (0, 0, 0, 0, 0).

The graph form will look as follows:

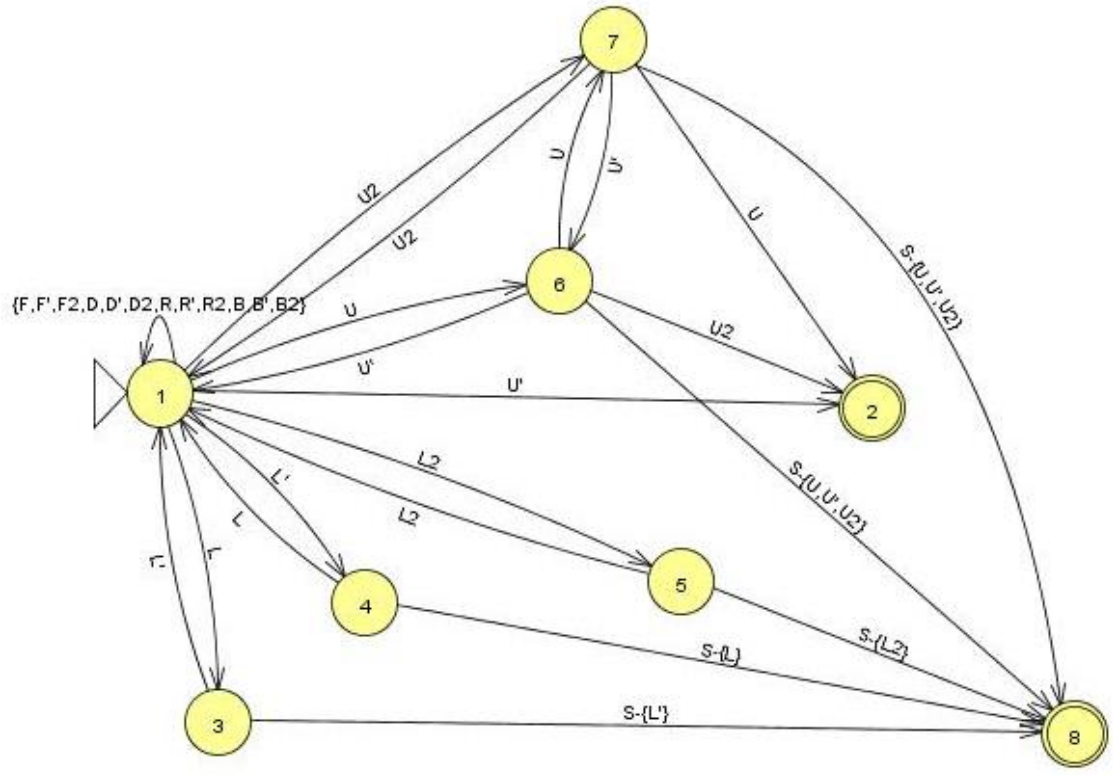

Figure 27 FSM for the sixteenth starting state from the list (1, 1, 3, 2, U)

17. (1, 2, 3, 3, U). The cubie is moved into the correct position (1, 2, 3, 1, U) by the following sequence: U2. If any other turn of the up face was made, it still will be possible to get the cubie in the desired place by additional turn of the up face. The turns of the most other faces won't influence the state of the Cube due to the fact that there is no solved cubies yet. Exception to this will be the turn of the back face, which will move the cubie that is being solved. In this case, user will be asked to undo the move. Therefore, states of the FSM will look like:

1. (1, 2, 3, 3, U);
2. (1, 2, 3, 1, U);
3. (1, 1, 2, 3, L);
4. (1, 3, 2, 3, R);
5. (1, 2, 1, 3, D);
6. (1, 1, 3, 2, U);
7. (1, 3, 3, 2, U);
8. (0, 0, 0, 0, 0).

The graph form will look as follows:

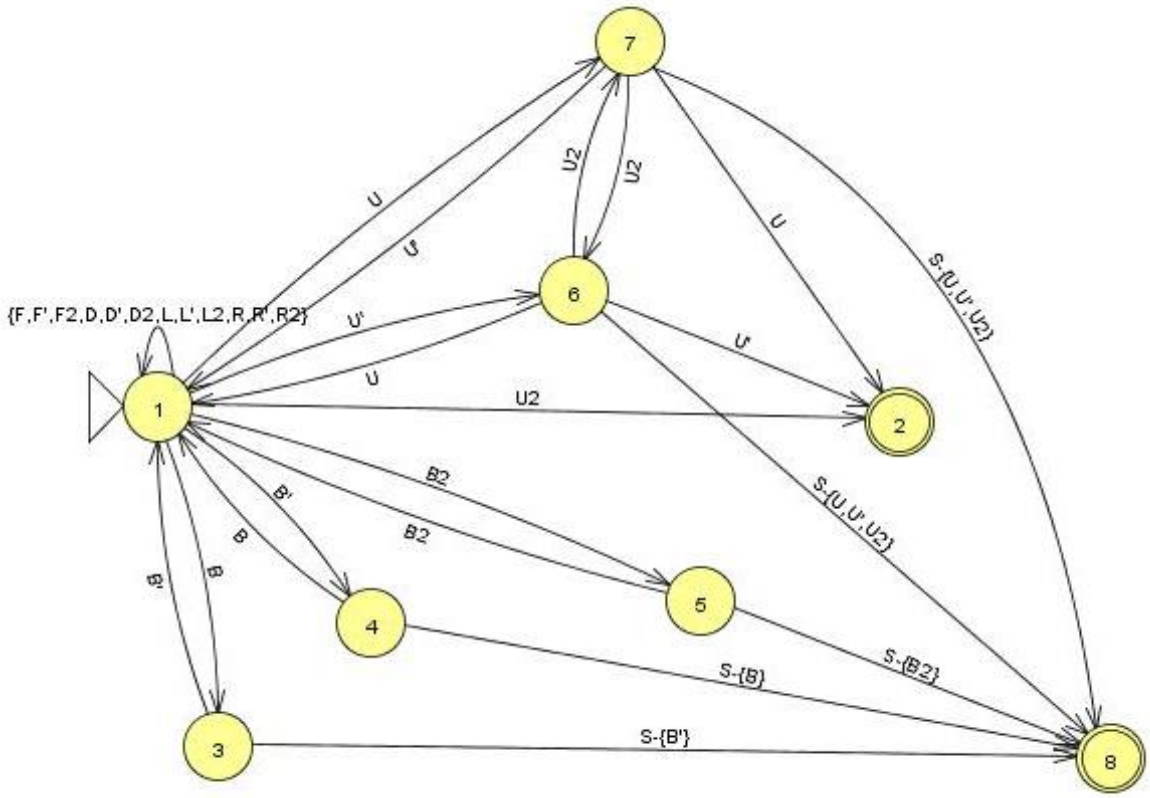

Figure 28 FSM for the seventeenth starting state from the list (1, 2, 3, 3, U)

21. (1, 2, 1, 3, D). The cubie is moved into the correct position (1, 2, 3, 1, U) by the following sequence: D2 F2. At the first move, if any other turn of the down face was made, it still will be possible to get the cubie in the desired place by additional turn of the down face. The turns of the most other faces won't influence the state of the Cube due to the fact that there is no solved cubies yet. Exception to this will be the turn of the back face, which will move the cubie that is being solved. In this case, user will be asked to undo the move. At the second move, any turn of the down face will basically return the FSM to the first move. Any turn of the front face other than F2 will require additional turn of this face. Turns of the rest of the faces won't influence the state of the cube in general. Therefore, states of the FSM will look like:

1. (1, 2, 1, 3, D);
2. (1, 2, 1, 1, D);
3. (1, 3, 2, 3, R);
4. (1, 3, 2, 1, L);
5. (1, 2, 3, 3, U);
6. (1, 1, 1, 2, D);
7. (1, 3, 1, 2, D);
8. (0, 0, 0, 0, 0);
9. (1, 2, 3, 1, U);
10. (1, 1, 2, 1, L);
11. (1, 3, 2, 1, R).

The graph form will look as follows:

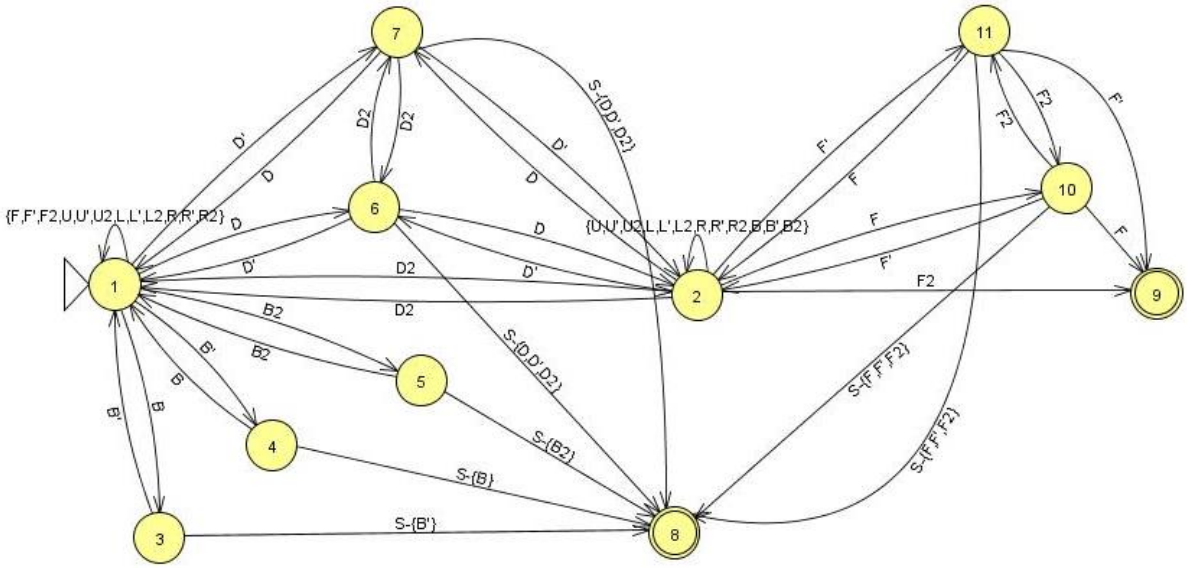

Figure 29 FSM for the twenty first starting state from the list (1, 2, 1, 3, D)